云图产品说明书

产品需求文档

修订历史

序号	版本	编写/修订说明	修订人	审核人	修订日期	备注
1	V1.0	云图产品文档	王雅琦、苗海芳、张民岗、彭成龙、江少波、金刚	吴航、张宇	2024 年 10 月 24 日	
2	V1.1	云图产品文档	王雅琦、苗海芳、张民岗、彭成龙、江少波、金刚	吴航、张宇	2024 年 10 月 31 日	
3	V1.2	云图产品文档	王雅琦、苗海芳、张民岗、彭成龙、江少波、金刚	吴航、张宇	2024 年 11 月 5 日	
4	V1.3	1. 修改 Guidance 透出类型的维护情况； 2. 增加 HDAir 相关接口的说明（【3.3.1.3.13 HD Air 接口】）； 3. 长实线 kLsl 增加部分字段，新增了 kSlope 的使用说明，新增了 KTraffcLightCtrl 类型的使用说明； 4. 新增了获取数据版本的两种方式的说明（【3.3.1.3.14 获取数据版本】） 5. 新增【 3.3.5 地图数据 SDK 常见问题】 6. 更正【3.3.1.3 数据引擎功能模块介绍】的子章节的序号； 7. 新增【3.3.1.3.15 获取车道数变化点】 8. 2025 年 01 月 13 日新增【3.3.1.3.16 用指定的数据版本号请求数据】	彭硕、赵天涯	江少波	2024 年 12 月 19 日 2025 年 01 月 13 日	文档标记 V1.3
5	V1.4	1. 补充数据引擎架构图 2. 新增数据引擎 状态码 10 鉴权失败 3.	彭硕	江少波	2025 年 03 月 15 日	文档标记 V1.4

一、产品概述

1.1 产品概述及目标

该文档是面向自动驾驶域的产品设计人员、研发人员提供的产品说明，用于进行产品功能的介绍、使用方法的说明，帮助产品、研发快速了解产品的功能。

1.2 阅读对象

该文档适用于产品、研发人员阅读。

1.3 术语与缩写解释

术语	英文释义	中文释义
SD Map	Standard Definition Map	传统导航地图
HD Map	High Definition Map	高精度地图
HAD Map	Highly Automated Driving Map	高度自动驾驶地图

二、整体架构

导航地图 SDK:

导航地图：从第三方导航域将导航路径的信息传输的自驾域，将导航提供的导航路径、定义道路 Link 信息等传输给的自动驾驶域的异源匹配的接口。

腾讯地图云:

激活鉴权服务：提供的 lic 激活、鉴权的服务，用于进程车辆注册、激活、认证等功能，对车辆的接入和数据下载的身份进行验证。
注：对应后续的 3.1 章节

异源匹配服务：提供第三方导航路径与腾讯地图的路径匹配服务；
注：对应后续的 3.2 章节

地图发布服务：提供地图数据发布服务，将天级/季度级别更新频率的数据发布到腾讯云上，供地图数据 SDK 下载读取使用。
注：对应后续的 3.3.3 章节

车端自驾域:

地图数据引擎 SDK：地图数据引擎 SDK，腾讯内置了数据更新接口、地图数据缓存、版本差分等功能，可以将地图数据通过 API 接口，提供给感知系统进行集成。

注：对应后续 3.3 章节

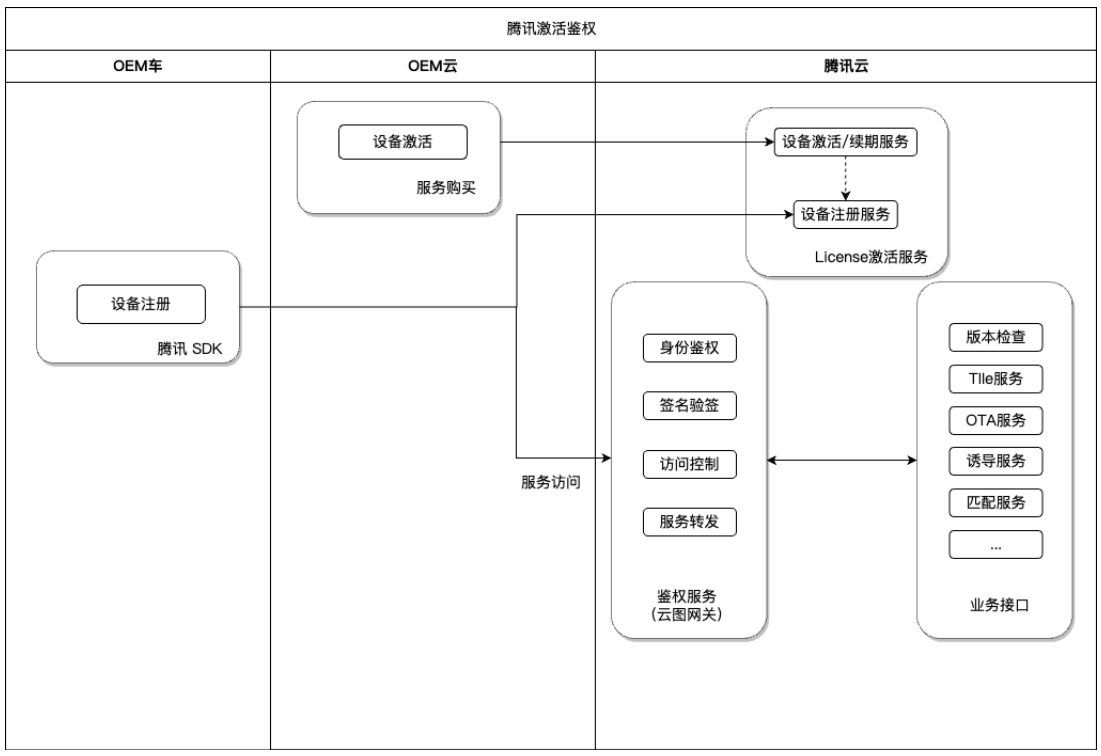
异源匹配 SDK：自动驾驶系统商，通过调用该接口，将第三方的导航路径与腾讯的 SD 路网进行匹配，形成与导航路径匹配的 link 串，并可以通过诱导进行数据的播发。

感知规划控制系统：系统商将传感器数据、地图数据进行融合感知，形成感知结果和规控的信号，进行车辆的控制。

三、详细功能模块

3.1 License 激活鉴权

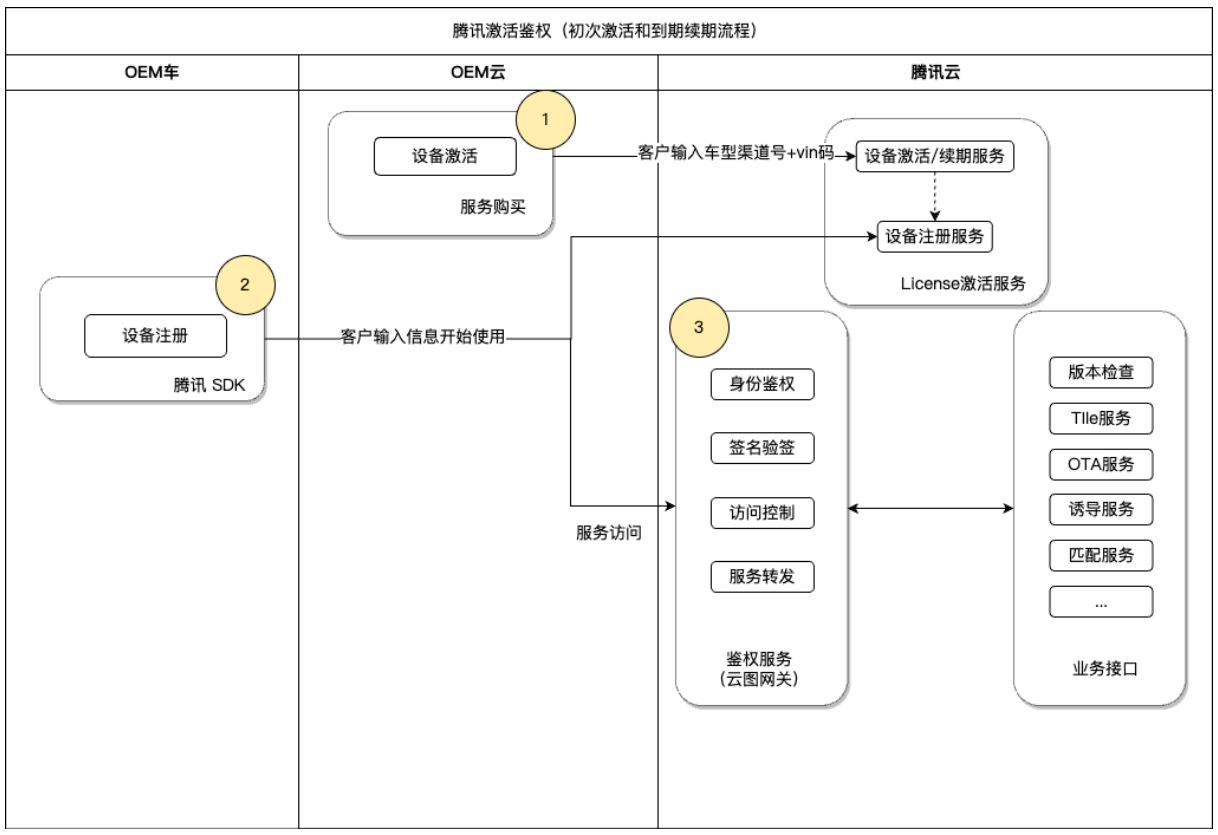
3.1.1 License 激活和鉴权的整体功能说明



2024/10/31 变更 1031：

初次流程：

腾讯智驾数据服务所有业务接口，在终端调用的时候都会进行鉴权，来控制访问是否合法，是腾讯对于接口服务的一种防盗手段，基于此，腾讯提供 Licence 激活鉴权服务能力，用来给 OEM 进行设备激活、车端注册，以及腾讯对服务访问的鉴权控制，首次激活和鉴权的整体流程如下所图所示，主要包含三步：



1. 设备激活，OEM 输入由 OEM 云端和腾讯云端进行对接。
   a. 供 OEM 客户进行下单购买车端设备访问云端数据的权限，只有成功激活过的设备，才可以成为正式的”用户“，即被腾讯认定为合法正式的设备，后续才可以取到”激活码”，如果不激活，就不可以使用腾讯智驾服务的接口服务，也即无法通过接口获取到地图数据。
   b. 激活通过接口方式实现，由 OEM 来调用，激活过程中通过接口参数控制为待激活的设备购买的服务/产品种类，服务时限范围，和计费模式。
   c. 后续 OEM 和腾讯之间的结算，也将基于激活的记录出账单。
   d. 同一次激活下单，可为多个设备激活。同一个设备，可多次下单激活，如果该设备没过期，则将转化未续费，如果过期了将再次激活。
   e. 除激活接口外，腾讯还提供激活业务相关服务接口，例如可通过激活订单查询历史激活记录，可通过设备 VIN 码查询该设备的激活记录、当前服务剩余时限范围，购买过的服务/产品种类等等信息。
2. 设备注册， 由腾讯车端 SDK 和腾讯云端进行对接。

a. 设备注册是用来获取腾讯为该设备分配的身份标识码和设备访问腾讯接口服务的 Token 令牌；

b. 只有激活过的设备且仍在服务期范围内的设备，才能够获取到身份标识码和 Token 令牌，如果没有激活过或已过期，将返回相关异常状态码；

c. 设备标识为腾讯生成的 DeviceId，非 OEM 车端设备的 VIN 码，后续访问腾讯接口服务均需携带该设备标识，用作鉴权；

d. Token 令牌为腾讯通过激活信息用算法生成的一个身份令牌，无法被篡改，无法被仿造。每个激活成功的车端设备都有一个唯一的令牌，即”一车一证“，Token 中包含了该令牌归属的设备身份标识，能访问腾讯的服务/产品种类（即能调用哪些接口），以及该令牌的有效期范围等等信息，后续访问腾讯接口服务均需携带该设备标识，用作鉴权。

3. 服务鉴权，由 OEM 车端（腾讯车端 SDK） 和腾讯云端对接。

a. 腾讯智驾数据服务所有业务接口都由”云图网关“统一接入，包括 Tile 服务、OTA 服务、异源匹配服务等，通过网关接入进行的接口调用，都会对调用者问的进行合法性校验，只有通过合法性校验，才允许访问智驾数据服务的接口。

b. 网关主要通过接口请求中携带的 Header 参数进行鉴权，鉴权的内容主要包含调用者身份、接口数据电子签名，以及业务的访问控制，只有均通过以上验证，请求才会转发到实际的业务接口上进行处理，业务接口处理完成以后再通过网关把响应参数返回到调用端。

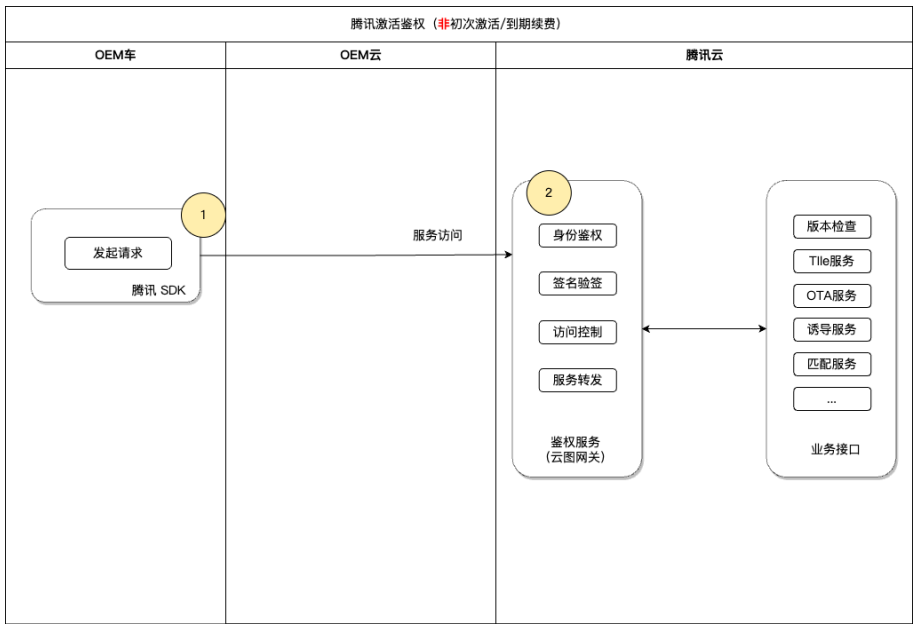
c. 服务鉴权的前置条件是调用接口服务的设备进行过了激活注册，否则则无法访问接口，接口返回的响应参数里会给出对应的状态码

d. 正常情况下车端的网关接入是由腾讯的车端 SDK 和腾讯的云端对外完成的，客户无需关注。OEM 云端如果有访问腾讯智驾服务接口的场景的话，需要和腾讯对接网关如何接入，后续章节中将由说明。

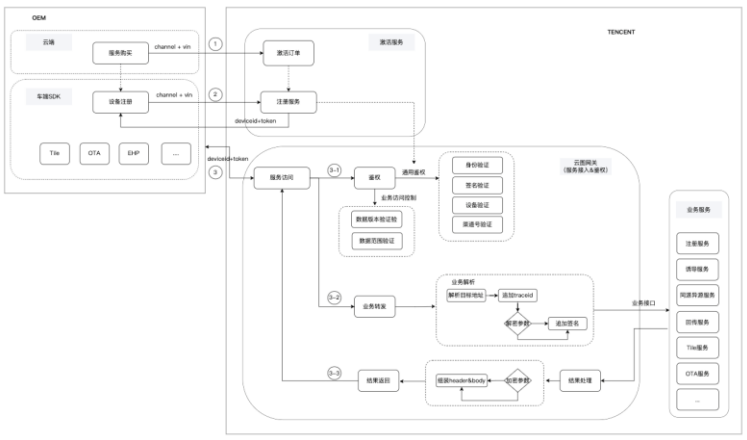
2024/10/31 变更 1031：

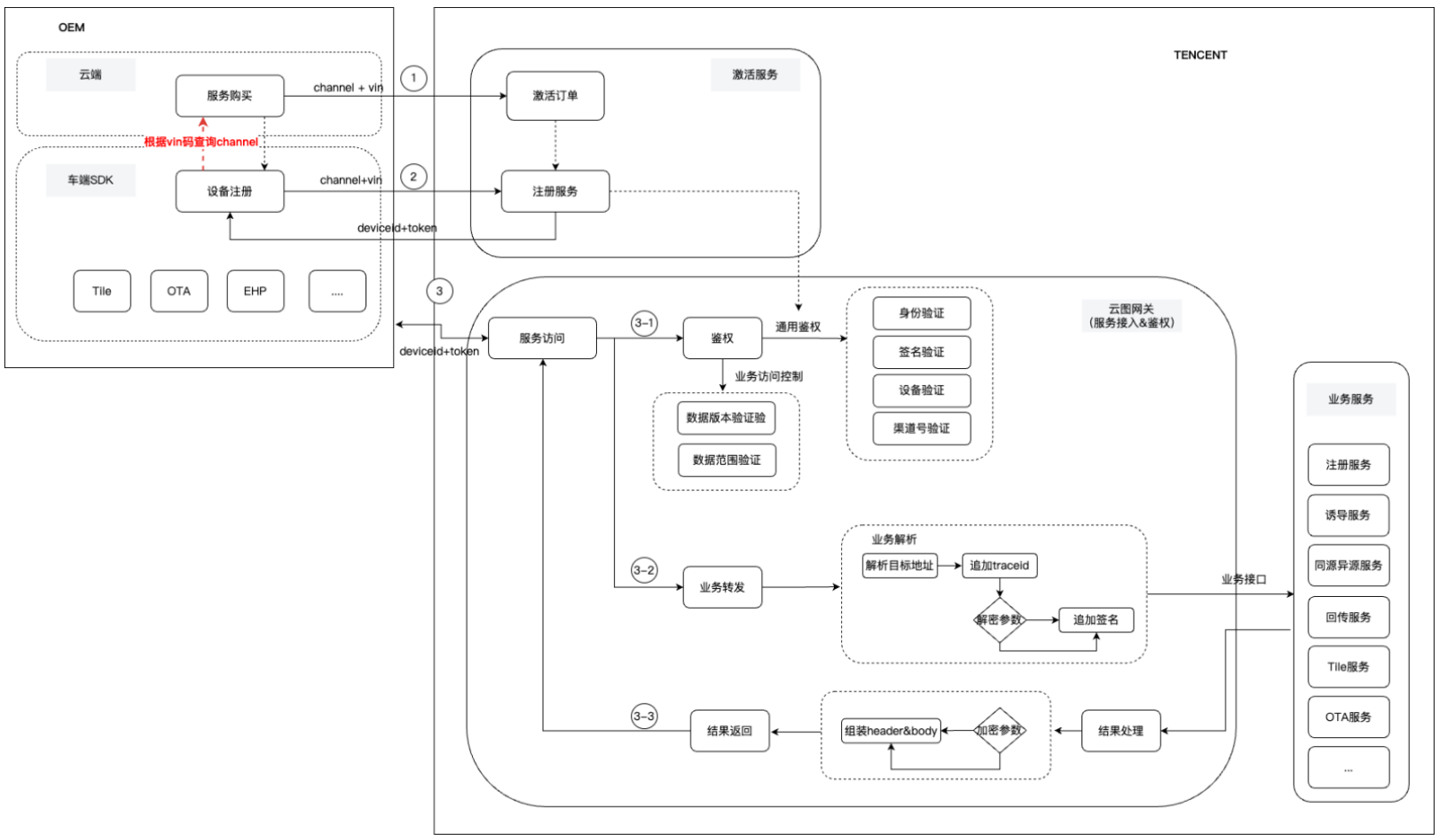
后续流程：

腾讯智驾数据激活和注册后，车端发起请求后，鉴权后直接获得对应的数据服务：



3.1.2 Licens 激活和鉴权的功能模块架构





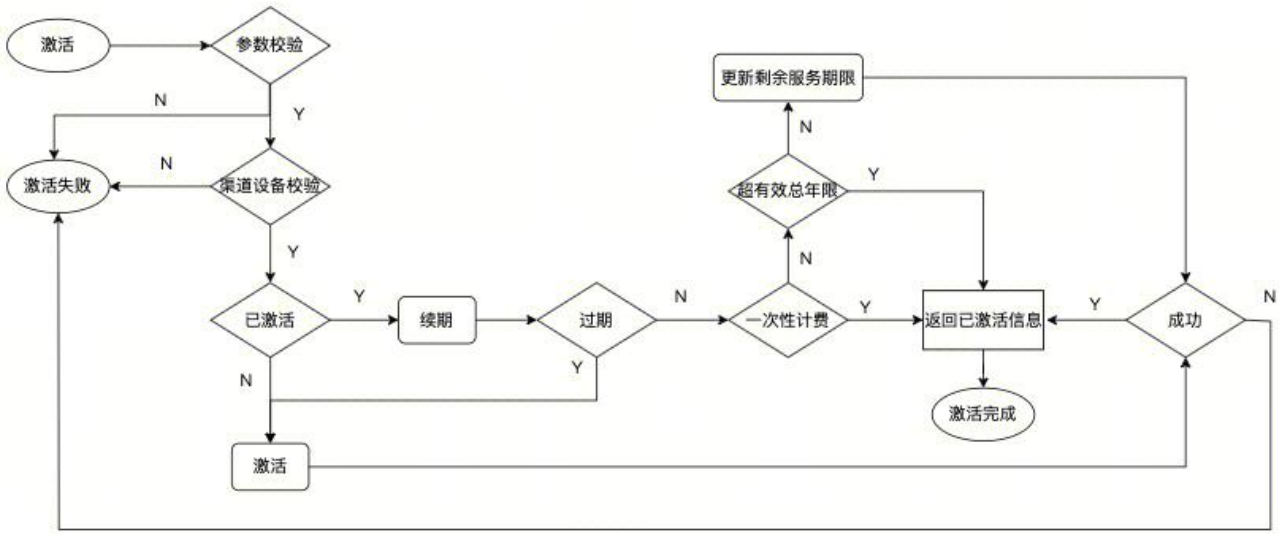
1. 客户先为自己待使用腾讯智驾数据服务的设备进行激活，即为该设备购买对应业务服务的合法使用期限，即”先购买“。
2. 客户在车端使用腾讯 SDK 调用腾讯智驾数据服务的接口的时候，SDK 里内置了设备注册的，获取到腾讯分配的身份标识和访问令牌，即“后使用”。
3. 1）设备顺利激活注册后，即可获得身份标识和访问令牌，从车端发起请求后，云端会进行鉴权和访问控制
    i. 鉴权主要验证访问着的身份信息，渠道号信息，设备信息进行合法性验证，另外通过 token 可知道该设备的令牌是否合法，是否匹配要访问的接口，是否过期等等，其次安全方面要求，会进行签名的验证，即判断该服务接口参数在传输过程中是否有被劫持修改。
    ii. 访问控制主要判断该请求要访问的数据内容是否在售卖范围之内，例如城市范围控制，网格范围控制，地图要素范围控制，数据版本控制等等。

    2）当前鉴权通过后网关即对该请求放行，允许访问后端的接口

    3）当后端接口有相关返回后，网关会再把接口的响应数据返回给车端 SDK，包括一些正常或异常响应数据，如果请求有异常，对应的接口服务里会返回对应的异常状态码，车端 SDK 会解析该状态并做对应的逻辑处理。

3.1.3 Licens 激活和鉴权的功能模块设计

3.1.3.1 激活/续费



【激活续费接入功能说明】

激活/续费是客户为自身设备购买腾讯数据服务的一种途径，该购买行为客户主发起，建议由云端发起，便于客户留存购买的下单记录，成为后续对账结算的依据。激活有设备的服务时间范围，可基于此生成访问令牌和续费；腾讯在收到激活请求后主要处理流程如下

1. 对接口参数进行校验，如果参数校验不通过，将不允许激活(激活失败)，返回”-1“的异常状态码(参数异常)，校验的主要内容有：
    a. 非空校验，如渠道号，VIN 码，购买的服务，客户订单编号等
    b. 参数值合法性校验，如购买的服务不存在，计费模式不存在或不支持等
    c. 参数数值相关校验，如激活时间超过一定的年限，订单中为待激活的设备数量等

2. 判断当前设备是否已激活，如果未激活过（无激活历史记录），则将对该设备进行激活，激活成功后将会给客户返回激活信息，激活完成。
3. 如果该设备激活过（有激活历史记录），则将看之前的激活是否已过期，如果过期，还将将进入激活流程。如果该设备之前的激活还未过期，则将计入续期流程。
4. 当进入续期流程后，首先会看客户为该设备购买的服务的计费模式，如果是一次性计费的，则终止续费流程，返回该设备之前的激活历史信息。
5. 如果续期流程中客户为该设备购买的服务的计费模式是按需计费，则会校验之前的激活剩余服务期限加当前续费的服务期限累加起来是否超过 30 年，如果超过 30 年，则将终止续费流程，返回该设备之前的激活历史信息。
6. 如果续费的计费模式是按需计费并且服务期间累加没有 30 年，则将进行为该设备需求的流程，续期完成后修改该设备的新的服务期限，并返回该设备的激活信息。续期完成。

【激活续费输入信息】

1. header 输入参数

   参考 ”服务网关接入 “，header 说明章节

2. body 输入参数

参数	说明	参数提供方	是否必须	默认值
channel	由腾讯分配的渠道号，用以识别客户身份	腾讯	是	
customerOrderId	客户购买数据服务的订单编号	OEM	是	
orderTimeUnit	订单数量对应的单位，如，年/月/日，枚举值：year/month/day	OEM	是	
orderCount	订单数量，和订单单位组合约定购买后的服务时限，如单位是年，数量是 1，即购买 1 年的服务 激活/续费操作设备累计有效剩余年限不允许超过 30 年	OEM	是	
billMode	计费模式，为设备激活而下单购买的计费方式，如一次性计费（永久有效），按需计费（根据实际需要购买对应的时限）等，枚举值：1（一次性计费），2（按需计费）	OEM	是	
product	产品标识，即购买腾讯的哪些产品服务，购买哪些服务只能访问哪些服务，没购买的不允许访问	OEM	是	
vids	客户车端 vin 码，单次下发可以批量为设备激活，可以传入小于 100 的设备数量	OEM	是	

【激活续费输出结果】

1. body 输出参数

参数	说明
errCode	激活操作接口返回的业务状态码，0 为成功，非 0 为失败，具体值参考下文 ”异常说明 “
errMsg	状态码对应的中文简要描述
orderId	腾讯订单编号，OEM 可以存，可以忽略
customerOrderId	客户购买数据服务的订单编号
orderTimeUnit	订单数量对应的单位，如，年/月/日，枚举值：year/month/day
orderCount	订单数量，和订单单位组合约定购买后的服务时限，如单位是年，数量是 1，即购买 1 年的服务 激活/续费操作设备累计有效剩余年限不允许超过 30 年
billMode	计费模式，为设备激活而下单购买的计费方式，如一次性计费（永久有效），按需计费（根据实际需要购买对应的时限）等，枚举值：1（一次性计费），2（按需计费）

product	产品标识，即购买腾讯的哪些产品服务，购买哪些服务只能访问哪些服务，没购买的不允许访问
vids	批量激活的设备信息
vid	客户车端 vin 码，单次下发可以批量为设备激活，可以传入小于 100 的设备数量
startTime	订单服务开始时间（当次激活或续费的开始时间）
endTime	订单服务结束时间

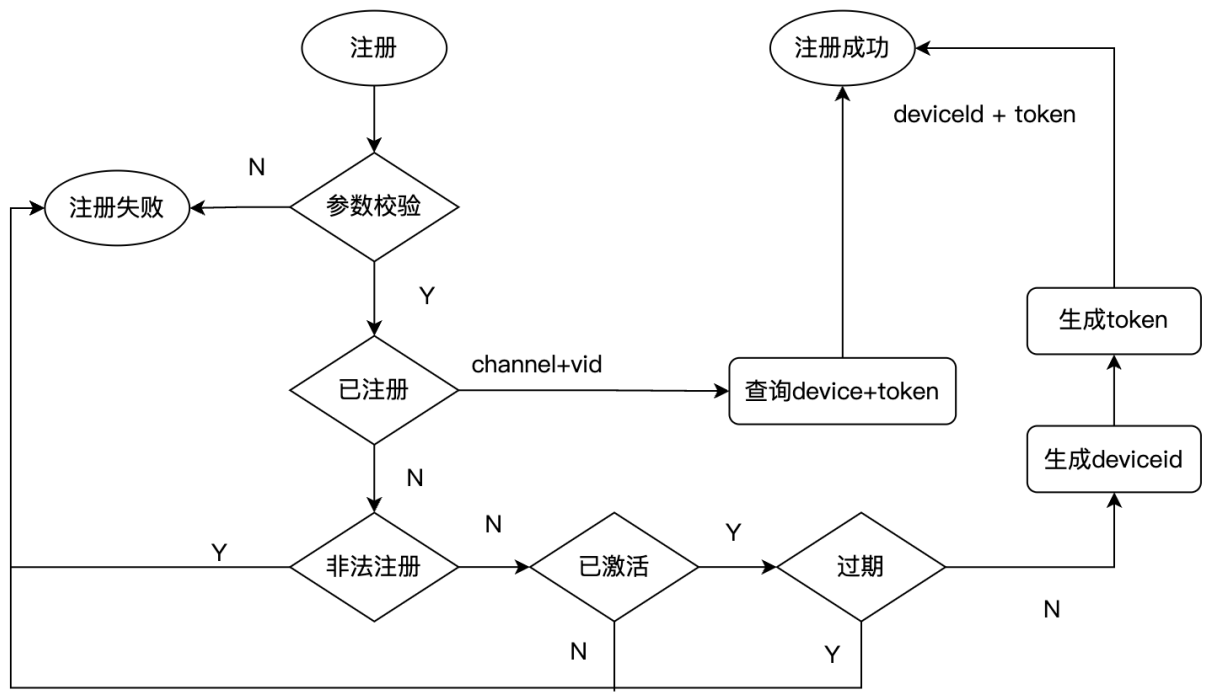
【示例说明】


```
{
  "errCode": 0,
  "errMsg": "操作成功",
  "data":{
     "orderId":"b2d104abc7c0543a81594d4b9700af71" ,
     "customerOrderId": "7d47fbbc-8c1a-4500-9d38-002826140286",
      "orderTime":"2024-08-25 23:57:23",
    "orderTimeUnit":"month",
    "orderCount":10,
    "billMode":1 ,
     "vids":[
          {
               "vid":"SIL_C01_13248119",
               "startTime":"2024-08-25 23:57:23",
               "endTime":"2025-08-24 23:59:59"
          }
     ]
  }
}
```


【异常说明】

状态码	异常类型	详情描述
0	成功	
-1	参数异常	
-57	激活设备相关异常	激活设备的数量等 设备已存在 设备不存在 等等
-58	激活订单相关异常	非法服务期限 订单已存在 订单不存在 等等
-9999	服务异常	

3.1.3.2 设备注册



【设备注册接入功能说明】

注册是腾讯车端 SDK 和腾讯云端交互的内部行为，用户不需要参与，仅是为了后续服务的鉴权需要，来获取腾讯为设备分配的设备标识和访问令牌。前置条件是客户已为该设备做过激活，即通过服务购买的接口下单激活该设备，腾讯在收到激活请求后主要处理流程如下

1. 接口参数校验，包括渠道号，设备 VIN 码，腾讯 License 码等，如果参数校验不通过，将不允许激活(激活失败)，返回”-1“的异常状态码(参数异常)。
2. 判断该设备是否已经注册过，如果注册过了，查询该设备的身份标识和访问令牌，返回给车端 SDK，流程结束。
3. 判断该次行为是否为非法注册，如果验证不通过将返回”-52“异常状态码，非法注册定义如下：
   a. 该设备是否被禁用，如当前有一些异常访问，被腾讯识别并禁用该设备
   b. 该次注册使用的 license 码是否已经用过，一个 license 码只能激活一个设备，不允许在多个设备上激活使用
4. 判断该设备是否激活过，如果没有激活过将不允许注册，返回对应的”-55“异常状态码。
5. 判断该设备以往的激活历史，如果已过期了，返回对应的”-55“异常状态码。
6. 以上校验或判断均通过后，将会为该设备注册，生成该设备的 Deviceid 和 Token，deviceid 即腾讯为该设备分配的身份标识，token 即腾讯为该设备生成的身份令牌。deviceid 和 token 后续客户在访问腾讯的智驾数据服务接口携带，用以鉴权的数据输入和数据依赖。

Token 采用 JWT 标准算法生成，算法因子包括渠道号，腾讯设备标识码，服务期限，服务产品等信息，当客户端来请求的时候，服务端即可通过解析 token，来判断该请求身份是否合法，包括渠道号信息、设备信息、服务期限、购买的服务产品等，来判断该请求是否允许，达到身份鉴权和服务鉴权，

JTW 算法输入的结构：

```
{
 "header":{
  "alg": "HS256",
  "typ": "JWT"
 }
 "body":{
  "channel":"CHANNEL", # 渠道号，客户标识
  "deviceid":"DEVICE", # 腾讯设备身份标识
  "devicetime":"DeviceTime", # 该设备激活后有效截止时间
  "product":"Product" # 客户为该设备购买的腾讯产品
 }
}
```

设备 TOKEN 返回客户端后，客户端妥善保存并在每次请求中携带。只有服务端持有单方面的加密密钥能够解开先前签发的 TOKEN 验证该 TOKEN 的有效性。同时 TOKEN 中编码设备信息（渠道、deviceid、vin 等），因 TOKEN 整体加密，该信息无法被篡改，服务端借助这些信息可以确保该 TOKEN 的身份来源。

【设备注册输入信息】

1. header 输入参数

   参考”服务网关接入“，header 说明章节

2. body 输入参数

参数	说明	参数提供方	是否必须	默认值
channel	由腾讯分配的渠道号，用以识别客户身份	腾讯	~~是~~否	
vid	客户车辆 vin 码，客户车端设备唯一标识码，针对同一台设备，激活和注册的输入信息里需要保持一致，否则腾讯将不会识别为同一台设备，即使激活过了，也会出现无法注册和无法访问腾讯智驾数据服务的接口	OEM	是	
uuid	腾讯车端 SDK 为该设备生产的唯一标识码	腾讯	~~是~~否	

【设备注册信息说明】

1. body 输出参数

参数	说明
errCode	激活操作接口返回的业务状态码，0 为成功，非 0 为失败，具体值参考下文”异常说明“
errMsg	状态码对应的中文简要描述
deviceId	腾讯设备 id，后续交互流程中鉴权使用，即身份标识
token	设备 token，后续交互流程中鉴权使用，即身份令牌

【示例说明】

```
{
  "errCode": 0,
  "errMsg": "操作成功",
  "deviceId":"3w5e0zejewikg",
  "token":"eyJEZXZpY2UiOiIzdzVlMTB0cndrdHRzIiwiQ2hhbm5"
}
```

【异常说明】

状态码	异常类型	详情描述
0	成功	
-1	参数异常	
-52	注册设备相关异常	设备非法（已被禁用） 设备非法（已被注册）
-55	注册设备未激活或激活已过期	
-9999	服务异常	

3.1.3.3 鉴权服务(云图网关)

腾讯智驾服务所有业务接口，全部通过云图网关接入，在此做统一做鉴权服务，以便控制访问的合法性，鉴权不通过将不允许访问业务接口，鉴权通过后才会放行。鉴权内容如下：

1. 身份鉴权

   即对调用着的身份进行识别，识别的依据即使是否有合法的腾讯分配的设备身份标识（deviced id）和访问令牌（token），鉴权服务在收到请求

后会判断该设备是否禁用，通过解密 token，判断该设备购买的服务是否过期，该设备是否和激活时候购买的客户信息一致，是否购买过当次调用的接口的产品服务等，鉴权不通过返回”407“http 状态码

2. 签名鉴权

签名主要是判断数据在客户端和服务端传输的过程中是否被篡改过，签名的实现是客户端和服务端分别对参数信息进行统一算法加密运算，看两次加密算法产生的结果是否一致，如果一致，说明数据在传输过程中没有被串改过，否则无法保证服务访问的真实性，鉴权不通过返回”407“http 状态码。签名的具体要求和实现方式参考本章节”签名说明“

3. 访问控制鉴权

访问控制主要对可以要访问的数据是否被允许，例如，客户 A 只购买了北京的 SD 数据，而此次接口访问确想调用上海的 SD 数据，将不被允许，鉴权不通过返回”407“http 状态码。访问控制鉴权还包括城市范围、地图要素范围、地图瓦片范围等。

以下为客户端通过网关访问智驾数据服务接口的说明

【鉴权服务功能说明】

1. 域名

开发环境域名：gateway.testsite.woa.com

内网测试环境域名：apigw-test.testsite.woa.com

公网测试环境域名：apigw-trial.map.tencent.com

预发环境域名：apigw-pre.map.tencent.com

正式环境域名：apigw.map.tencent.com

2. 请求信息

路径:【与目标业务服务一致】，代理转发:
请求类型:【与目标业务服务一致】
传参方式:【与目标业务服务一致】

3. 签名说明

header 参数集：{tx-reqtype, tx-channel, tx-deviceid, tx-timestamp, tx-traceid}

参数拼接规则：将参数集中参数构造成字典，并以 key 按字典序排序，将字典 key、value 直接拼接组成字符串,例如 tx-channel[CHANNEL]tx-deviceid[DEVICE]tx-reqtype[REQTYPE]tx-timestamp[TIMESTAMP]tx-traceid[TRACEID]

签名算法：md5 （ String sign = DigestUtils.md5DigestAsHex(signString);）

4. 请求 header 参数

tx-reqtype	string	是	请求类型，由腾讯统一分配 account：激活服务 register：注册服务 tile：tile 服务 guideadapter：诱导服务 datamanager：ota 服务
tx-channel	string	是	渠道，由腾讯统一分配
tx-deviceid	string	是	设备标志，由设备激活流程获取
tx-token	string	是	身份令牌，由设备激活流程获取
tx-proto-version	int	否	协议版本号，默认为 0，客户自定义
tx-timestamp	long	是	请求当时的时间戳（毫秒）

tx-traceid	string	否	请求标记，客户自定义
tx-sign	string	是	见上一节”签名说明“

5. 请求 body 参数

【目标业务自定义】

6. 响应 body 参数

【目标业务自定义】

【鉴权服务输入信息】

鉴权服务 即网关非实体业务接口，无独立的 body 参数输入，具体的 body 参数由不同的业务接口定义。Header 参考以上 Header 说明

【鉴权输出信息说明】

鉴权服务 即网关非实体业务接口，无独立的 body 参数输出，具体的 body 参数由不同的业务接口定义

【示例说明】

header 示例说明

	Key	Value
☑	Content-Type	application/json
☑	tx-reqtype	register
☑	tx-channel	10023202204
☑	tx-deviceid	9godlb4uamtc
☑	tx-proto-version	0
☑	tx-timestamp	1721896317025
☑	tx-traceid	658ab488-0803-4a6e-8510-a9436c14c9bd
☑	tx-sign	144857b5606f949934d0dcbd72b3f14c

【异常说明】

HttpCode	HttpMsg	说明	出现场景
200	OK	成功	网关转发成功（不代表业务逻辑正常）
407	PROXY_AUTHENTICATION_REQUIRED	鉴权失败	tx-deviceid、tx-channel、tx-token、tx-sign 等鉴权逻辑未通过
502	BAD_GATEWAY	网关错误	请求参数 tx-reqtype 错误导致无法找到目标下游服务，目标下游服务不可达，网关服务内部

3.2 异源匹配服务和诱导信息

3.2.1 异源匹配和诱导信息整体功能说明

在自动驾驶系统中，座舱域和自驾域的功能和数据来源可能存在差异。例如，座舱域可能采用非腾讯的方案，而自驾域则使用腾讯的数据。在这种情况下，如何有效地利用腾讯的数据来支持自动驾驶的决策和导航变得尤为重要。通过异源匹配服务，可以将腾讯的数据与其他数据源进行整合，从而实现更高效的导航和决策支持。

3.2.1.1 异源匹配的功能

1. 数据整合与映射：异源匹配服务的核心功能是将不同来源的数据进行整合和映射。在自动驾驶中，使用腾讯的数据时，可以通过分析路线的形状点（必须要素）以及道路等级、道路属性和道路名称（更优要素），将这些信息与当前的驾驶环境进行匹配。这种映射不仅可以恢复出准确的导航路线，还能确保系统在不同数据源之间的无缝切换。

2. 路径恢复：通过对路线形状点的匹配，系统能够重建出车辆在行驶过程中应遵循的路径。这对于确保自动驾驶车辆能够准确地遵循预定路线至关重要，尤其是在复杂的城市环境中，能够有效避免因数据不一致而导致的导航错误。

3.2.1.2 诱导信息的功能

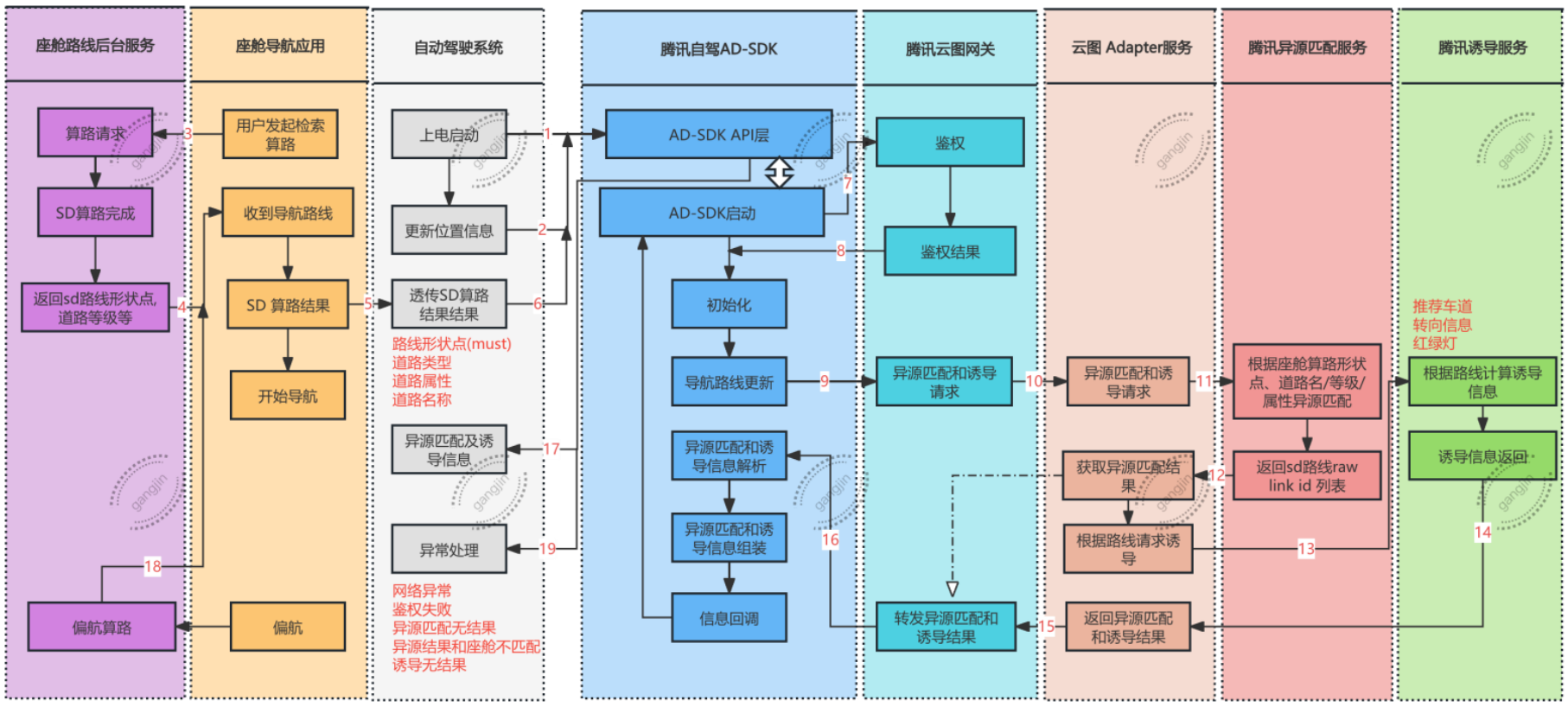
1. 辅助决策：诱导信息在自动驾驶中扮演着至关重要的角色，特别是在变道和转向时。通过提供推荐车道、转向信息和红绿灯位置等数据，系统能够在复杂的交通环境中做出更为精准的决策。这些信息可以帮助自动驾驶系统判断何时变道、何时减速或加速，从而优化行驶策略。

2. 减少计算负担：在自动驾驶过程中，实时计算和决策是非常复杂的任务。通过提供诱导信息，自驾系统可以减少对环境的实时计算需求。例如，推荐车道信息可以直接告诉系统在特定情况下最优的行驶车道，从而减少对周围环境的全面分析，提升反应速度。

综上所述，异源匹配和诱导信息在自动驾驶系统中发挥着重要的作用。异源匹配通过整合和映射不同来源的数据，确保导航路径的准确性和可靠性；而诱导信息则为自动驾驶系统提供了关键的决策支持，帮助系统在复杂的交通环境中做出更为安全和高效的行驶决策。通过这两者的结合，自动驾驶系统能够在不同的数据环境中实现更高的智能化和自动化水平，从而提升整体的驾驶安全性和用户体验。

3.2.2 异源匹配和诱导信息整体架构

具体架构如下所示：



注：座舱导航后台服务、座舱导航应用、自动驾驶系统为外部系统，不包含在腾讯系统范围内。

具体模块的信息介绍如下：

1. 腾讯自驾 AD-SDK

客户端 AD-SDK 是自动驾驶系统中的集成组件，负责与自驾系统进行交互。其主要功能包括：

参数接收与处理：客户端 SDK 能够接收来自自驾系统的调用参数，这些参数可能包括当前车辆位置、目标位置、行驶状态等信息。SDK 会对这些参数进行解析和处理，以便后续的网络请求。

网络请求：在接收到参数后，SDK 会根据需要向云图网关发起网络请求，获取异源匹配和诱导数据。此过程需要确保网络连接的稳定性和数据传输的安全性。

数据回调：一旦获取到所需的数据，客户端 SDK 会将这些数据回调到自驾系统的调用方。这一过程需要确保数据的完整性和准确性，以便自驾系统能够基于这些数据做出正确的决策。

2. 腾讯云图网关

云图网关是整个系统的中枢，负责设备鉴权和消息转发。其主要功能包括：

设备鉴权：云图网关负责对接入的客户端进行身份验证，确保只有经过授权的设备才能访问系统资源。这一过程有助于保护系统的安全性，防止未授权访问。

消息转发：云图网关的核心功能是将客户端 SDK 发来的请求消息转发到相应的服务（如云图 Adapt 服务）。它不参与具体的业务逻辑处理，而是充当一个中介，确保消息能够高效、准确地传递。

监控与日志记录：云图网关可以记录所有的请求和响应日志，以便后续的监控和故障排查。这一功能有助于提高系统的可维护性和可靠性。

3. 云图 Adapter 服务

云图 Adapter 服务是系统中负责业务逻辑处理的模块，主要功能包括：

通用能力服务：云图 Adapter 服务提供了一系列通用能力，如数据格式转换、请求调度等，确保不同模块之间能够顺利协作。

数据请求与整合：该服务负责向异源匹配服务和诱导服务发起请求，获取所需的数据。它能够根据业务需求，灵活地组合和整合来自不同服务的数据，以满足客户端 SDK 的请求。

业务逻辑处理：云图 Adapt 服务可以根据具体的业务需求，进行复杂的逻辑处理。例如，它可以根据当前的交通状况和车辆状态，动态调整请求的参数，以获取最优的数据。

4. 腾讯异源匹配服务

异源匹配服务是自动驾驶系统中负责路径恢复的模块，主要功能包括：

路径恢复：根据输入的路线计算的形状点（必须要素），该服务能够恢复出车辆的导航路线。这一过程需要考虑道路的实际情况，以确保路径的可行性和安全性。

数据融合：除了形状点，该服务还可以利用道路等级、道路属性和道路名称等信息，进一步优化恢复的导航路线。这种数据融合能力有助于提高导航的准确性和可靠性。

5. 腾讯诱导服务

诱导服务是自动驾驶系统中负责提供辅助信息的模块，主要功能包括：

诱导信息计算：该服务负责计算与车辆行驶相关的诱导信息，包括推荐车道、转向信息等。这些信息对于自动驾驶系统的决策至关重要，能够帮助系统在复杂的交通环境中做出更为安全和高效的行驶决策。

规控应用：诱导服务的输出信息将直接应用于自动驾驶的规控系统，确保车辆在行驶过程中能够遵循交通规则和安全标准。这一功能有助于提升自动驾驶的安全性和合规性。

通过以上模块的协同工作，整个自动驾驶系统能够高效地获取、处理和应用所需的数据，从而实现安全、可靠的自动驾驶体验。

【主要流程说明】

第 1 步：自动驾驶系统上电后启动腾讯 AD-SDK

在自动驾驶系统通电后，系统会自动启动腾讯 AD-SDK。此时，SDK 会进行初始化，包括加载必要的配置文件、建立网络连接和准备接收来自自驾系统的参数。这一过程确保 SDK 能够在后续操作中正常工作。

第 2 步：自驾系统更新实时位置信息到腾讯自驾 AD-SDK

如果需要分段发送诱导信息，自动驾驶系统会定期更新其实时位置信息（如 GPS 坐标、速度、方向等）到腾讯 AD-SDK；如果只需要一次性获取路线上所有的诱导信息，不需要实时更新自动驾驶系统位置。

第 3 步：用户在座舱导航 APP 上通过检索发起算路

用户在座舱导航应用中输入目的地信息并发起算路请求。导航应用会通过后台服务进行路线计算，考虑当前的交通状况、道路限制和用户偏好等因素，以生成最佳路线。

路线服务后台接收到用户的算路请求后，进行复杂的算法计算，生成路线结果。计算结果包括 SD 路线的形状点、道路等级、道路属性等信息，这些信息将用于后续的导航和路径匹配。

第 4 步：路线服务后台返回 SD 路线的形状点，道路等级等信息到座舱导航应用

计算完成后，路线服务后台将生成的路线信息（如形状点、道路等级等）返回给座舱导航应用。此时，导航应用会将这些信息准备好，以便后续传递给自动驾驶系统。

第 5 步：座舱导航应用把路线上算路结果，通过跨域通信传送给自动驾驶系统

座舱导航应用通过跨域通信机制，将算路结果（包括路线的形状点和相关信息）传送给自动驾驶系统。这一过程需要确保数据的完整性和安全性，以便自动驾驶系统能够准确接收和处理这些信息。

第 6 步：自动驾驶系统把算路结果传给腾讯自驾 AD-SDK

自动驾驶系统接收到算路结果后，将其传递给腾讯 AD-SDK。此时，SDK 会对接收到的路线信息进行解析，以便后续的鉴权和数据请求。

第 7 步：腾讯自驾 AD-SDK 向腾讯网关发起鉴权

腾讯 AD-SDK 在接收到算路结果后，首先需要进行设备鉴权。SDK 会向腾讯云图网关发起鉴权请求，确保当前设备具备访问后续服务的权限。

第 8 步：腾讯云图网关将鉴权结果返回给腾讯自驾 AD-SDK

腾讯云图网关在接收到鉴权请求后，会进行身份验证，并将鉴权结果（成功或失败）返回给腾讯 AD-SDK。如果鉴权成功，SDK 将继续进行后续的数据请求；如果失败，SDK 将处理鉴权失败的情况。

第 9 步：鉴权结果通过后，腾讯自驾 AD-SDK 把收到的导航路线向腾讯云图网关发起网络请求

在鉴权成功后，腾讯 AD-SDK 会将接收到的导航路线信息（如形状点、道路等级等）通过网络请求发送给腾讯云图网关，以便后续的数据处理和服务调用。

第 10 步：腾讯云图网关透传请求到腾讯云图 adapter 服务

腾讯云图网关接收到来自 AD-SDK 的请求后，会将该请求透传到腾讯云图 Adapter 服务。此时，网关仅作为消息的转发者，不参与具体的业务逻辑处理。

第 11 步：腾讯云图 adapter 服务向腾讯路线匹配服务发起路线匹配请求

腾讯云图 Adapter 服务在接收到请求后，会向腾讯路线匹配服务发起请求，要求根据传入的导航路线信息进行路径匹配。这一过程涉及到对路线的分析和处理，以确保匹配结果的准确性。

第 12 步：腾讯路线匹配服务，把恢复出来的导航路线 link id 的列表返回给腾讯云图 adapter 服务

腾讯路线匹配服务根据请求进行计算，恢复出导航路线，并生成相应的 link ID 列表。该列表包含了与导航路线相关的道路信息，随后将其返回给腾讯云图 Adapter 服务。

第 13 步：腾讯云图服务通过恢复出来的路线，向腾讯诱导服务计算路线的诱导信息

腾讯云图 Adapter 服务在接收到 link ID 列表后，会利用这些信息向腾讯诱导服务发起请求，计算与该导航路线相关的诱导信息。这些信息包括推荐车道、转向信息等，旨在为自动驾驶系统提供更为详细的行驶指导。

第 14 步：腾讯诱导服务把诱导信息返回给腾讯云图 adapter 服务

腾讯诱导服务完成计算后，将生成的诱导信息返回给腾讯云图 Adapter 服务。此时，Adapter 服务将拥有完整的导航路线和诱导信息，以便后续的结果整合。

第 15 步：云图 adapter 服务将路线匹配结果和诱导结果返回给腾讯云图网关

腾讯云图 Adapter 服务将路线匹配结果和诱导信息整合后，返回给腾讯云图网关。此时，网关将持有完整的导航信息，准备将其下发给 AD-SDK。

第 16 步：腾讯云图网关将所有结果下发给腾讯自驾 AD-SDK

腾讯云图网关将整合后的导航路线和诱导信息下发给腾讯 AD-SDK。此时，SDK 会对接收到的数据进行解析和处理，以便将其传递给自动驾驶系统。

第 17 步：腾讯自驾 AD-SDK 把路网和诱导信息组装信息之后发送给自动驾驶系统，诱导信息主要包括转向信息、转向距离，推荐车道三部分

腾讯 AD-SDK 在接收到导航路线和诱导信息后，会将这些信息进行组装，形成最终的路网和诱导信息。组装后的信息包括转向信息、转向距离和推荐车道等，随后将其发送给自动驾驶系统，以便系统能够根据这些信息进行安全、有效的行驶决策。

第 18 步：如果座舱发生了偏航，只要重复以上步骤即可

在行驶过程中，如果座舱发生偏航或其他状态变化，系统可以重复以上步骤，重新计算和获取最新的导航信息和诱导数据，以确保自动驾驶系统始终能够根据最新的环境和状态做出反应。

【输入信息说明】

名称	作用	数据结构	说明	参数提供方	备注
TargetType	定义请求类型	SD=0	SD/HDAir 异源匹配数据。	客户提供参数	该参数决定客户希望把导航路径与那个地图产品匹配，默认为 SD 数据
		HD=1	HD 异源匹配数据。	客户提供参数	
		SD_GUIDE=2	SD/HDAir 异源匹配数和诱导信息。	客户提供参数	
Point	定义坐标点	uint32_t lng	经度，GCJ02 坐标系的经度，放大 10^7 倍。	客户提供参数	必填
		uint32_t lat	纬度，GCJ02 坐标系的纬度，放大 10^7 倍。	客户提供参数	必填
OriginalLink	定义道路 Link 信息	int32_t length	道路长度，单位为米。	客户提供参数	选填
		int32_t linkType	道路类型：0-普通道路，1-航道，2-隧道，3-桥梁，4-高架桥。	客户提供参数	选填
		int32_t roadClass	道路等级：0-高速公路，1-国道，2-省道，3-县道，4-乡公路，5-县乡村内部道路等。	客户提供参数	选填
		int32_t roadType	道路类型：1-主路，2-路口内部道路，3-JCT 道路等。	客户提供参数	选填
		std::string roadName	存储道路的名称。	客户提供参数	选填
		int32_t coor_idx	当前 link 在路线中形点的起始位置。	客户提供参数	选填
		int32_t coor_num	当前 link 在路线中形点的数量。	客户提供参数	选填
NaviPath	定义导航路径	std::string path_id	输入的路径标识符。	客户提供参数	必填
		uint32_t length	路线长度，单位为米。	客户提供参数	必填
		std::vector<Point> coors	当前导航路线的所有坐标点。	客户提供参数	必填
		std::vector<OriginalLink> links	绑定结果的各个 link 信息。	客户提供参数	必填
MMRequest	定义请求结构	TargetType type	指定请求的目标类型为字段“定义请求类型“信息	客户提供参数	必填
		std::vector<NaviPath> path	最多支持三条路径。	客户提供参数	必填

【输出信息说明】

名称	作用	数据结构	说明	备注
TPIDStatus	定义 TPID 状态	EXISTED = 0	已存在。	
		NOT_EXIST = 1	不存在。	
		BAD_ID = 2	数据异常。	
TPIDInfo	定义 TPID 信息	TPIDStatus status	raw link id 对应的 tpid 绑定状态。	
		uint32_t tile_id	13 级瓦片，tpid 同瓦块内唯一。	
		uint32_t id	link 在瓦片内的索引值；无数据版本概念；从 1 开始，不存在时返回 0，最大值 4294967295。	
TxLink	定义原始链接信息	uint64_t raw_link_id	raw link id。	
		TPIDInfo tpid	raw link id 对应的 tpid。	
		uint32_t dir	方向：0 顺方向，1 逆方向。	
		std::vector<Point> points	raw link 的形点串。	
		int32_t length	raw link 长度，单位厘米。	
BindPoint	定义绑定点信息	int32_t status	绑定状态：0 表示成功，非 0 表示匹配失败，以下字段无意义。	
		uint64_t raw_link_id	raw link id。	

		uint32_t dir	方向：0 顺方向，1 逆方向。	
		uint64_t timestamp	对应输入点的时间戳。	
		Point point	该点对应到 raw link 上的绑定点的经纬度。	
		uint32_t offset	绑定点在 link 上的偏移量，取值在[0-100]之间。	
		uint32_t distance	点到绑定 link 的距离，单位为厘米。	
LinkGroup	定义链接组信息	int32_t status	状态：0 表示匹配成功的区间；1 表示未匹配的区间；2 表示匹配失败的区间；3 表示多路径匹配区间。	
		std::vector<TxLink> linkinfo	绑定结果的各个 link 信息。	
		int32_t bindpoint_idx	当前 linkgroup 绑定的输入点序列位置。	
		int32_t bindpoint_num	当前 linkgroup 绑定的输入点数量。	
		Point start_loc	linkgroup 起始位置。	
		Point end_loc	linkgroup 终点位置。	
		uint32_t length	linkgroup 长度，单位厘米。	
		uint32_t confidence	linkgroup 置信度，只有存在多路径匹配区间的 linkgroup 有意义；单位%。	
Result	定义异源匹配结果信息	std::string path_id	输入对应的 path id。	
		std::vector<LinkGroup> link_groups	绑路结果，数组中每个值为一组连续 link。	
		std::vector<BindPoint> bind_points	每个输入点对应的绑路结果。	
		std::string data_version	预留。	
RoutePos	定义路线位置	Point pos	坐标点。	
		uint32_t coor_start	点在路线点串的起始索引。	
		uint32_t offset_length	点相对 coor_start 对应形状点的偏离距离，单位米。	
RGIntersection Result	定义转向信息结果	uint64_t raw_link_id	raw link id，暂不支持。	
		int inter_type	机动点类型，转向模型。	
		int accessory_info	辅助动作。	
		RoutePos enter_pos	进入机动点路口位置。	
		RoutePos leave_pos	离开机动点路口位置。	
RGLaneInfoRe sult	定义车道信息结果	uint64_t raw_link_id	raw link id，暂不支持。	
		std::vector<uint32_t> arrow	车道箭头类型信息。	
		std::vector<uint32_t> flag	车道线高亮车道信息。	
		std::vector<uint32_t> property	车道线车道属性信息。	
		std::vector<uint32_t> recommend	推荐车道标识。	
		std::string active_time	车道生效时间，eg,07:00-09:00｜17:00-19:00，暂不支持。	
		RoutePos pos	坐标点。	
RGRedLightRe sult	定义红绿灯结果	uint64_t raw_link_id	raw link id，暂不支持。	
		RoutePos pos	坐标点。	
Route	定义导航路线	std::vector<Point> route_points	抽稀后的道路导航路线形状点。	
		std::vector<RGIntersectionResult> rg_intersection_result	导航转向信息。	
		std::vector<RGLaneInfoResult> rg_lane_info_result	推荐车道信息。	
		std::vector<RGRedLightResult> rg_red_light_result	红绿灯信息。	
RouteGuide	定义路线引导	std::vector<std::string> route_id	route id，同一 path id 可能对应多段 route。	
		std::vector<Route> routes	路线信息。	
RouteGuideRe sponse	定义路线引导响应	int32_t errcode	错误代码。	
		std::string msg	消息。	

		std::vector<RouteGuide> result	诱导结果。	
MMGuideResponse	定义 MM 引导响应	MMResponse mm_response	异源匹配返回结果。	
		RouteGuideResponse route_guide_response	诱导信息返回结果，和异源匹配返回结果一一对应。	
MMGuideResponse	定义 MM 引导响应	MMResponse mm_response	异源匹配返回结果。	
		RouteGuideResponse route_guide_response	诱导信息返回结果，和异源匹配返回结果一一对应。	

【配置信息说明】

配置项	作用说明	默认值	备注
channel	渠道号，由腾讯项目提供，一般一个车型一个渠道号，没有配置正确的渠道号无法获取异源匹配和诱导能力，服务根据不同的渠道号下发不同的定制信息	无	腾讯提供
device_id	设备唯一标识，由 SDK 调用方提供，主要用于设备的鉴权和调查问题是日志信息查询	1234match	调用方提供
https_timeout	https 请求超时时间,单位毫秒	3000	调用方可配置
data_root_path	数据存放路径,包括日志,鉴权数据等	./data	调用方可配置
tca_log_level	日志级别,可以为 0-verbose 详细的日志信息 1-debug 调试日志 2-info 一般信息 3-warning 警告日志 4-error 错误 5-fatal 致命错误 6-no_log 不输出日志	4	调用方可配置日志的级别
log_name_prefix	日志文件名前缀,如果为空,则使用默认配置，日志文件名格式: tencent_tca_20240912_202517_977.log.gz	tencent_tca_	tencent_tca 为日志前缀，调用方可配置前缀
log_storage_time_limit	日志存储时间限制,单位为天,如果小于等于 0 表示使用默认配置	30 天	调用方可配置
log_storage_total_size_limit	日志存储大小的最大限制，单位为 Byte，如果小于等于 0 表示使用默认配置	200MB	调用方可配置
log_storage_single_size_limit	单个日志文件大小限制，单位为 Byte，如果小于等于 0 表示使用默认配置	5MB	调用方可配置

【异常错误码】

错误码	msg	说明	备注
0		成功	
10001	timeout	网络超时	
10002	unknown error	http 未知错误	
10003	cancel	http 请求撤销错误	
30001	auth error, please check channel and device id	鉴权错误	
1	coords too short	形点数量不可小于 3 个	服务返回
1	distance between points exceeds limit: 5000	形点间最大距离不可超过 5km	服务返回

1	request navi path length exceeds limit:150000	路线最长不超过 150km	服务返回
5XX	server error	超时等错误	服务返回

【接入文档和示例说明】

详见附录一；

3.3 地图数据 SDK

地图数据 SDK 包含两部分：数据引擎模块以及离线数据管理模块，在此章节分别介绍。

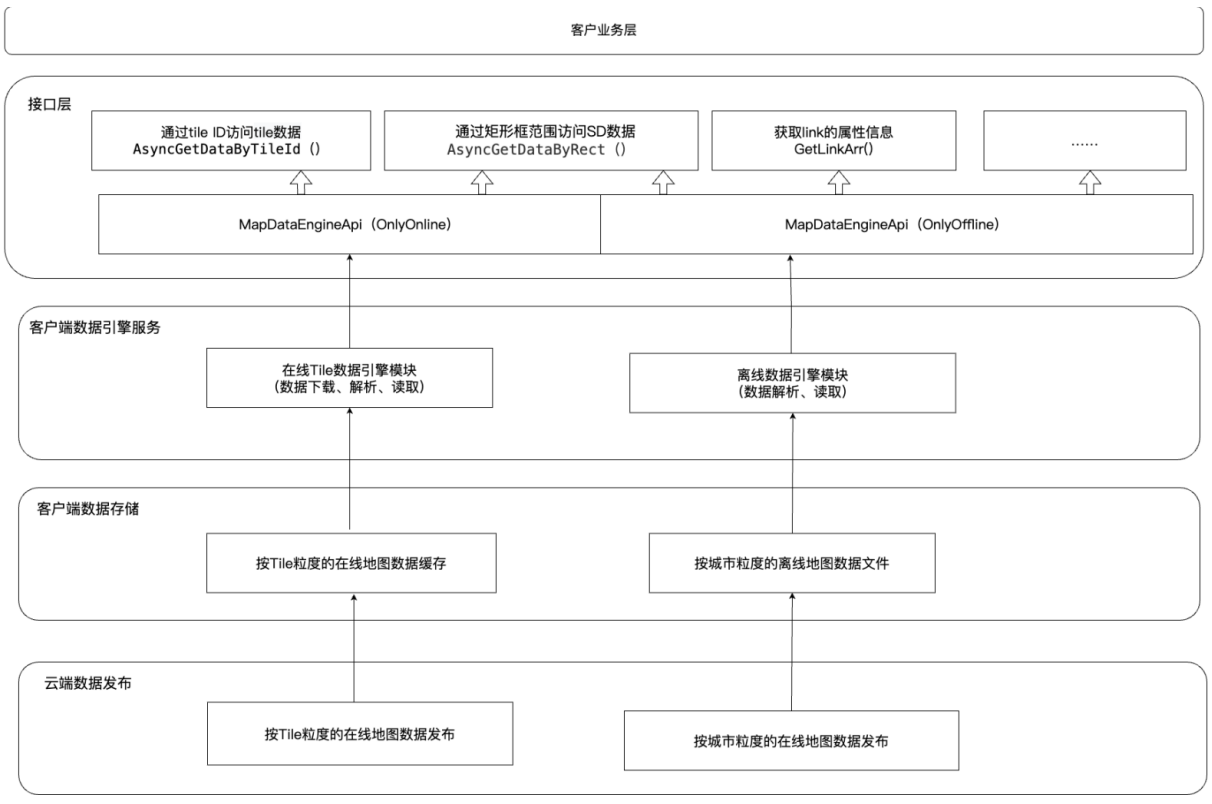
3.3.1 数据引擎模块

3.3.1.1 数据引擎整体功能说明

数据引擎是一套面向自动驾驶业务的地图数据查询接口，可以同时支持 SD、HD、HD Air 等多种规格的数据查询；提供通过 tile id 查询 tile 瓦片数据块，以及通过自定义矩形框查询特定矩形范围的数据块两种查询方式；同时支持纯在线查询云端最新地图数据，以及纯离线查询本地预装的 NDS 数据文件的地图数据两种工作模式；

3.3.1.2 数据引擎功能模块架构

具体的模块架构如下图所示：



数据引擎功能模块架构 V1.4 补充

接口层：提供查询地图数据的接口方法，包括通过 tile ID 访问 tile 数据、通过矩形范围访问范围数据、获取 link 的属性信息等接口；支持在线和离线两种模式的 API，分别访问在线的 tile 数据和离线预装的 NDS 地图数据；

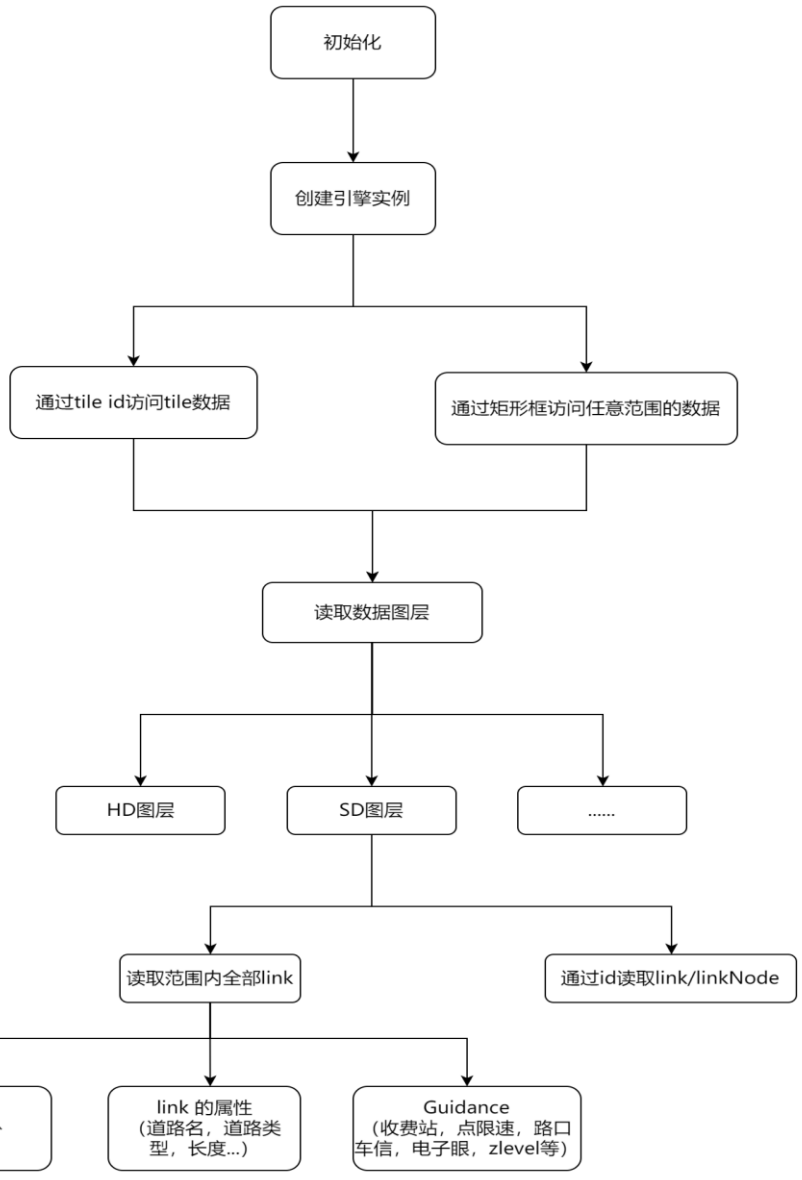
车端数据引擎服务：客户端服务层，负责数据的下载、读取和解析等核心逻辑；

车端数据存储：包含在线模式按 tile 下载缓存的 NDS 地图数据、离线模式下预装的 NDS 地图数据；

注：由于数据鲜度不一致，在线模式下载的数据和离线模式预装的数据不能混用；

数据发布服务：云端的数据更新服务，支持按 tile 粒度的地图数据下载更新（天级更新/月度级更新），同时支持按城市粒度对预装的离线地图数据进行 OTA 更新，具体见 3.3.3 章节；

3.3.1.3 数据引擎功能模块介绍



注：具体见附件二；

初始化：使用引擎功能接口前，需要先完成初始化；初始化的接口和参数说明参考 3.1 章节；

创建引擎实例：在初始化之后，创建一个数据引擎的实例，为后续的数据操作做准备。具体方法参考 3.2 章节；

读取地图数据块：可以通过两种方式读取一整块范围的地图数据，

1. **通过 tile id 访问 tile 数据：**数据引擎可以通过输入 tile id 来读取单个 tile 范围（大约 2.5KM*2.5KM）的数据。具体方法参考 3.3 章节。
2. **通过矩形框访问任意范围的数据：**另一种数据访问方式，通过输入矩形框的左下右上坐标来读取自定义范围的数据。具体方法参考 3.4 章节。

读取数据图层(HD/SD)：在读取到整块范围的数据后，需要再读取具体的数据图层，主要包含 SD（HA）图层和 HD 图层。具体方法参考 3.5 章节。

读取范围内全部 Link：在 SD 图层中，数据引擎提供接口读取该范围内所有的 Link 数据。具体方法参考 3.6 章节。

通过 id 读取 Link/Link Node：可以通过要素 id 来读取单个要素详情数据。具体方法参考 3.7 章节。

link 的拓扑：读取 Link 数据后，可以进一步获取 Link 的拓扑结构。具体方法参考 3.9 章节。

link 的属性：拿到 link 后，可以获取 Link 的属性信息，如道路名、道路类型、长度等。具体方法参考 3.8 章节。

Guidance：通过 link，还可以获取相关联的 Guidance 信息，包括收费站、点限速、路口车信、电子眼、zlevel 等。具体方法参考 3.10 章节。

3.3.1.3.1 引擎初始化：InitGlobalConfig

【功能说明】

使用数据引擎需先完成初始化

【参数输入】

参数	说明	参数提供方	是否必须	默认值
data_root_path	数据存储路径，主要用于存储在线下载的数据和离线 NDS 数据	客户自己设置路径	是	默认存储路径：data

channel	由腾讯分配的渠道号，用于数据更新和设备鉴权	腾讯分配	是	
deviceId	车辆的唯一识别码，由集成方获取并设置，用于鉴权，一般为 vin 码或者 vin 码的 hash 值，长度不能超过 32 位	客户提供参数	是	
http	集成方注入的网络接口实现，主要目的是避免依赖冲突，同时能灵活应对自动驾驶域控的网络条件，demo 中有样例代码可以参考	客户提供参数	是	
path	数据引擎的日志存放路径	客户提供参数	是	
max_cache_size_mb	在线 tile 粒度的数据缓存存储空间大小，默认 600M，外部可在 200M~5G 之间自由设置；当缓存的数据超过设置的大小时，数据引擎内部会自动清理缓存；	客户提供参数	否	600M
use_self_signed_cert	是否使用腾讯的 https 证书和域名，默认为使用，可以解决域控系统 https 证书缺失和证书更新的问题；腾讯签名的证书有效期 100 年；	客户提供参数	否	true
log_name_prefix	数据引擎生成的日志文件名前缀，可自定义，默认前缀为 nerd_, 日志文件名格式: 前缀_时间戳 log.gz	客户提供参数	否	nerd_
log_storage_time_limit_param	日志存储的时间限制，默认值为 30 天，超过时间引擎会自动清理日志	客户提供参数	否	30 天
log_storage_max_size_limit_param	日志存储的空间大小限制，默认为 64M，超过大小会自动清理日志	客户提供参数	否	64M
size_log_size_limit_param	单个日志文件大小限制，默认为 200M, 超过大小会重新生成一个日志文件	客户提供参数	否	200M

【输出结果】

无

【示例代码】

```
    //初始化日志，传入日志路径，日志文件名前缀，日志存储时间限制，日志存储空间限制，单个日志文件大小限制
     nerd::api::InitLogger("./log", /*日志路径*/
            false, "tencent_nerd_", /*日志文件名前缀*/
            std::chrono::hours(24*30),/*日志存储时间限制*/
            200 * 1024 * 1024,/*日志存储最大空间限制*/
            5 * 1024 * 1024);/*单个日志文件大小限制*/

    // 引擎初始化需要先调用，启动时调用一次就行
    nerd::api::GlobalConfig global_config;
    global_config.data_root_path = "./data";  // 数据存储路径
    global_config.channel = "POC";// 渠道号，用于设备鉴权
    global_config.deviceId = "testDeviceId";//车辆的唯一识别码，长度不能超过 32，用于鉴权
    global_config.http = std::make_shared<nerdDemo::HttpImpl>();//网络请求的接口实现类，demo 中有参考代码

    nerd::api::InitGlobalConfig(global_config);
```

3.3.1.3.2 创建引擎实例：CreateEngine

【功能说明】

在访问数据前，需要创建不同类型的引擎实例，用于读取不同类型的数据。创建引擎实例时，需明确需要访问的数据图层（SD 或者 HD）和数据来源（在线数据或者离线预装的 NDS 数据）；

【参数输入】

参数	说明	参数提供方	是否必须	默认值
user_id	用于标识使用方的 id，自定义的字符串类型。请根据业务模块负责内容赋值容易区分的名字	客户提供参数	是	unknow
build_block_id	需要读取的数据类型，例如 SD 数据，HD 数据等。类型是 MapDataBuildingBlockID，详见 MapDataBuildingBlockID 表。	客户提供参数	是	当前默认为 SD 数据，参数为 kLANE
preference	数据请求模式，在线模式，离线模式，注意自动驾驶应用场景不支持在线优先或者离线优先等模式。类型是：DataPreference，详见 DataPreference 表。	客户提供参数	是	当前默认为在线模式，参数为：kOnlineOnly
wait_ms	请求超时时间，如果请求的数据量大，建议设置大一点	客户提供参数	否	5000 ms
online_status_callback	在线 tile 数据版本发生变化时回调通知	腾讯定义，客户实现	否	unknow

MapDataBuildingBlockID 的其他常见枚举值：

枚举值	说明
kROUTE	道路数据层（SD）和轻图数据层（HA）
kLANE	高精数据层（HD）
……	……

DataPreference 的其他枚举值（自动驾驶地图只支持 kOfflineOnly 和 kOnlineOnly 两种模式）：

枚举值	说明
kOfflineOnly	纯离线模式：只读取本地的离线数据
kOnlineOnly	纯在线模式：只读取在线数据（含在线缓存）
……	……

【输出结果】

创建成功时，返回引擎实例的指针（MapDataEngineApi），创建失败时，返回空指针；

创建失败的原因主要是没有先完成初始化；

【示例代码】

```
// 创建各个数据类型图层对应的引擎
static auto create_engine =
    [](nerd::api::MapDataBuildingBlockID build_block_id) -> std::unique_ptr<nerd::api::MapDataEngineApi> {
  nerd::api::APIConfig api_config;
  api_config.user_id = "building_block:" + std::to_string(static_cast<int32_t>(build_block_id));
  api_config.build_block_id = build_block_id;
  api_config.preference = nerd::api::DataPreference::kOnlineOnly;//这里只推荐在线模式，或者离线模式，其他的在线优先等模式可能存在数据版本不一致的问题
   return nerd::api::CreateEngine(api_config);
};

auto lane_api = create_engine(nerd::api::MapDataBuildingBlockID::kLANE); //创建获取 HD 数据的引擎实例
auto route_api = create_engine(nerd:api::MapDataBuildingBlockID::kROUTE);//创建获取 SD 数据的引擎实例
```

3.3.1.3.3 通过 tile id 访问 tile 数据：AsyncGetDataByTileId

【功能说明】

查询特定 tile 的数据。腾讯的 SD 数据是按照 NDS 13 级 tile 来切分，HD 数据是按照 NDS 15 级 tile 来切分；

【参数输入】

参数	说明	参数提供方	是否必须	默认值
param	请求参数，具体见下面 GetMapDataBatchParam 表	客户通过坐标调用接口口后，提供参数	是	
callback	请求结果回调	腾讯定义，客户实现	是	

GetMapDataBatchParam：按 tile id 查询的请求参数详情

参数	说明	参数提供方	是否必须	默认值
tile_id_list	要查询的 tile id 列表，数据引擎提供了单独的接口可以通过坐标计算当前位置所在的 tile id，详情参考 nerdapi 文档	客户提供参数	是	
feature_set_map	附加图层属性，例如 SD+ 的车道数数据，非标准的数据，需要特别申请才能访问	客户提供参数	否	

【输出结果】

字段	说明
ret	包含返回状态码和描述信息，状态码参考下面的状态码表
req	本次结果的请求参数，参考上面 GetMapDataBatchParam 参数表
rsp	请求结果，包含了请求的 tile 数据块，参考下面 GetMapDataBatchResult 表

GetMapDataBatchResult，tile 请求结果数据体：

字段	说明
data_cubes	tile 数据块列表，和请求参数中的 tile_id_list 一一对应

is_online_result	当前返回的数据是在线 tile 数据（包含本地已缓存的 tile 数据），还是离线 NDS 文件中的数据

【异常说明】

状态码：

场景	错误枚举	错误码	定义	备注
接口错误码（nerd::api::error_code）	kOk	0	接口调用正常时返回 0; * 备注: 非 0 表示返回结果不可靠;	
	kError	1	接口调用其他异常时反馈错 误 、生成 url 请求失败	
	kInputParameterE rror	2	调用接口传入参数不符 合预期时返回	Input parameter error
	kNetworkError	3	在线下载遇到网络错误 时返回 、鉴权失败	Network error
	kVersionNotMatch	4	数据加载出现版本不匹 配时返回	Version not match
	kDataChecksumError	5	在线下载数据校验失败 时返回	
	kParseDataError	6	解析数据错误时返回	
	kWriteDataError	7	写磁盘遇到错误时返回	
	kWriteDataError	8	调用接口已经过期或不 维护的接口时返回	
	kBeyondUserWait Time	9	请求处理时间超过用户 设定最大等待时间	

【示例代码】

```
nerd::api::GeoCoordinate point{121.499721, 31.239846};
 //通过坐标计算所在的 13 级 tile id
auto route_tile_id = nerd::api::utils::GetTileIdByGeoCoordinate(point,13);

nerd::api::GetMapDataBatchParam route_batch_param;
route_batch_param.tile_id_list.insert(route_tile_id);
auto route_tile_callback = std::make_shared<TileCallback>();
route_api->AsyncGetDataByTileId(route_batch_param, route_tile_callback);
// 等待结果
route_tile_callback->Wait();

auto result = route_tile_callback->GetResult();
auto dataCubes = result->data_cubes;
for(auto datacube : dataCubes){
   auto sdLinkLayer = datacube->GetSDLinkLayer(); //读取 SD 数据图层
   getSDMapElement(sdLinkLayer);
}
```

3.3.1.3.4 通过矩形框访问任意范围的数据：AsyncGetDataByRect

【功能说明】

通过输入矩形框范围，查询范围所覆盖的地图数据；

【参数输入】

参数	说明	参数提供方	是否必须	默认值
param	请求参数，具体见下面 GetMapDataByRectParam 表	客户提供参数	是	
callback	请求结果回调	腾讯定义，客户实现	是	

GetMapDataByRectParam：按矩形框范围请求的参数详情

参数	说明	参数提供方	是否必须	默认值
level	tile 切分级别，sd 是 13 级（nerd::api::MapLevel::kLEVEL_13），hd 是 15 级（nerd::api::MapLevel::kLEVEL_15）	客户提供参数 根据选择的数据类型定义	是	默认为 sd 数据，则为 13 级；
rect	要查询的矩形范围。rect.p0 是矩形左下角的经纬度，rect.p1 是矩形右上角的经纬度。	客户提供参数	是	
feature_set_map	附加图层属性，例如 SD+ 的车道数数据，非标准的数据，需要特别申请才能访问	客户提供参数	否	

【输出结果】

字段	说明
ret	包含返回状态码和描述信息，状态码参考下面的状态码表
req	本次结果的请求参数，参考上面 GetMapDataBatchParam 参数表
rsp	请求结果，包含了请求的 tile 数据块，参考下面 GetMapDataBatchResult 表

GetMapDataBatchResult，tile 请求结果数据体：

字段	说明
data_cube	tile 数据块
is_online_result	当前返回的数据是在线 tile 数据（包含本地已缓存的 tile 数据），还是离线 NDS 文件中的数据

状态码表：

场景	错误枚举	错误码	定义	备注
接口错误码（nerd::api: :error_code ）	kOk	0	接口调用正常时返回 0; * 备注: 非 0，返回结果不可靠;	
	kError	1	接口调用其他异常时反馈错 误 、生成 url 请求失败	
	kInputParameterE rror	2	调用接口传入参数不符合预期时返回	Input parameter error
	kNetworkError	3	在线下载遇到网络错误 时返回 、鉴权失败	Network error
	kVersionNotMatch	4	数据加载出现版本不匹配时返回	Version not match
	kDataChecksumEr ror	5	在线下载数据校验失败时返回	

	kParseDataError	6	解析数据错误时返回	
	kWriteDataError	7	写磁盘遇到错误时返回	
	kWriteDataError	8	调用接口已经过期或不维护的接口时返回	
	kBeyondUserWait Time	9	请求处理时间超过用户设定最大等待时间	
	kAuthError（V1.4 新增）	10	鉴权错误	

【示例代码】

```
nerd::api::GeoCoordinate point{121.499721, 31.239846};
//以某一点为中心，半径 500 米，构建一个矩形框
nerd::api::Rect rect = nerd::api::utils::BuildRectByLength(point,500);

nerd::api::GetMapDataByRectParam rect_param;
rect_param.level = nerd::api::MapLevel::kLEVEL_13;//获取 SD 数据(地图缩放层级 13 层)
rect_param.rect = rect;
auto rect_callback = std::make_shared<RectCallback>();
route_api->AsyncGetDataByRect(rect_param, rect_callback);
// 等待结果
rect_callback->Wait();

auto result = rect_callback->GetResult();
if(result->data_cube != nullptr){
  auto sdLinkLayer = result->data_cube->GetSDLinkLayer();
  getSDMapElement(sdLinkLayer);
}
```

3.3.1.3.5 读取数据图层：GetHDLaneLayer()，GetSDLinkLayer()

【功能说明】

通过 tile 数据块对象（类型 IDataCube），读取其中相应的图层，SD 和 HA 数据层或者 HD 高精数据层。

【参数输入】

无

【输出结果】

字段	说明
IHDLaneLayer	获取 HD 高精数据层
ISDLinkLayer	获取 SD 和 HA 数据层

【异常说明】

无

【示例代码】

```
auto result = route_rec_callback->GetResult();
if (result->ret.ret == 0){
   if(result->data_cube != nullptr) {
      std::unique_ptr<IDataCube> data_cube = result->data_cube;
      auto hdLaneLayer = result->data_cube->GetHDLaneLayer();
      auto sdLaneLayer = result->data_cube->GetSDLinkLayer();
   }
}
```

```
auto result = route_tile_callback->GetResult();
if (result->ret.ret == 0){
  auto dataCubes = result->data_cubes;
  for(auto datacube : dataCubes){
    auto sdLinkLayer = datacube->GetSDLinkLayer();
    auto hdLinkLayer = datacube->GetHDLaneLayer();
  }
}
```

3.3.1.3.6 读取全部 Link：GetLinkArr

【功能说明】

获取数据图层里所有的 link 数据。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	false

【输出结果】

字段	说明
std::vector<ILinkConstPtr>	道路 Link 集合

【异常说明】

无

【示例代码】

```
nerd::api::ISDLinkLayer * sdLinkLayer = data_cube->GetSDLinkLayer();
if(sdLinkLayer != nullptr){
  auto links = sdLinkLayer->GetLinkArr();
  std::cout << "sdLinkLayer->GetLinkArr().size(): " << links.size() << std::endl;
  for(const auto& link:links) {
    ...
  }
}
```

3.3.1.3.7 通过 id 读取 Link/LinkNode: GetLinkByID，GetLinkNodeByID

【功能说明】

根据 Link/LinkNode 的 ID，获取对应的 Link 对象，或者 LinkNode 对象（道路 Link 的端点）。

【参数输入】

对于 GetLinkByID：

参数	说明	参数提供方	是否必须	默认值
link_id	请求参数，类型为 LinkIDType。具体见下面 LinkIDType 表	客户提供参数	是	

LinkIDType 类型:

参数	说明	参数提供方	是否必须	默认值
tile_id	tile 瓦片 id	客户提供参数	是	
link_id	link 在瓦片内唯一 id	客户提供参数	是	

【输出结果】

字段	说明
ILinkConstPtr	link 对象。详细属性见 ILinkConstPtr 表。

ILinkConstPtr：Link 对象

函数或字段	说明
TPIDType GetTPID()	获取永久 ID
LinkAttribute& GetAttribute()	获取 Link 的通用属性
LinkExtAttribute& GetExtAttribute()	获取 Link 的扩展属性
std::vector<IRestrictionConstPtr> GetRestrictions()	获取 Link 的交通限制信息
std::vector<IDLinkConstPtr> GetNext(LinkDirection direction, bool with_indoor = false)	获取 Link 的下游 Link 列表
std::vector<IDLinkConstPtr> GetPrevious(LinkDirection direction, bool with_indoor = false)	获取 Link 的上游 Link 列表
std::vector<IDLinkConstPtr> GetNextWithinIntersection(LinkDirection direction, bool with_indoor = false)	获取 Link 的下游 Link 列表，包含交叉点内 Link、室内道路
std::vector<IDLinkConstPtr> GetPreviousWithinIntersection(LinkDirection direction, bool with_indoor = false)	获取 Link 的上游 Link 列表，包含交叉点内 Link、室内道路
std::shared_ptr<std::vector<Coordinate>> &GetGeometry()	获取 Link 的几何形状
int64_t GetRawID()	获取原始 ID
const std::shared_ptr<const std::vector<LinkCurvature>> GetLinkCurvature()	获取当前 Link 的曲率数据
std::vector<std::shared_ptr<const IGuidanceInfo>> GetGuidance(GuidanceType type)	获取当前 Link 的 Guidance 信息
NodeIDType GetStartNodeID()	获取当前 Link 的起点 NodeID
NodeIDType GetEndNodeID()	获取当前 Link 的终点 NodeID
.....	.....

【异常说明】

无

【示例代码】

```
auto node= sdLinkLayer->GetLinkNodeByID(link->GetStartNodeID());
for (auto linkId: node->GetOutLinkIDs()){
  auto link = sdLinkLayer->GetLinkByID(linkId);
}
```

对于 GetLinkNodeByID：

参数	说明	参数提供方	是否必须	默认值
node_id	请求参数，类型为 NodeIDType。具体见下面 NodeIDType 表	客户提供参数	是	

NodeIDType 类型：

参数	说明	参数提供方	是否必须	默认值
tile_id	tile 瓦片 id	客户提供参数	是	
node_id	node 在瓦片内唯一 id	客户提供参数	是	

【输出结果】

字段	说明
ILinkNodeConstPtr	node 对象。详细属性见 ILinkNodeConstPtr 表。

ILinkNodeConstPtr：道路 Link 的端点 LinkNode 对象。

函数或字段	说明
NodeIDType GetNodeID()	获取 NodeID
TPIDType GetTPID()	获取永久 ID
CrossFlag GetCrossFlag()	获取交叉口路口标识
bool GetTrafficLightFlag()	获取红绿灯标识。有红绿灯返回 true，否则返回 false
ILinkNodeConstPtr GetMainNode()	获取主 Node，主点包括完整的路口拓扑，当 CrossFlag 为 kSubNode 时返回主 Node，其他情况下返回 nullptr
std::vector<ILinkConstPtr> GetEnterElements()	获取进入的所有 link 元素集合
std::vector<ILinkConstPtr> GetOutElements()	获取出度的所有 link 元素集合
std::vector<ILinkConstPtr> GetEnterElementsCross()	获取所在路口的所有进入 link 元素集合
std::vector<ILinkConstPtr> GetOutElementsCross()	获取所在路口的所有出度 link 元素集合
Coordinate GetPosition()	Link Node 坐标。注意 Coordinate 类型通过 ToGeo()才能转成经纬度对象。
std::vector<LinkIDType> GetEnterLinkIDs()	获取进入的所有 linkID 集合
std::vector<LinkIDType> GetOutLinkIDs()	获取出度的所有 linkID 集合
std::vector<LinkIDType>	获取所在路口的所有进入 linkID 集合

GetEnterLinkIDsCross()	
std::vector<LinkIDType> GetOutLinkIDsCross()	获取所在路口的所有出度 linkID 集合
std::vector<IConditionConstPtr> GetGuidance(GuidanceType type)	当前 LinkNode 的 Guidance 信息
NodeIDType GetMainNodeID()	获取主 Node ID，~~主点包括完整的路口拓扑，~~当 CrossFlag 为 kSubNode 时返回主 Node ID，其他情况下返回空值

【异常说明】

无

【示例代码】

```
auto links = sdLinkLayer->GetLinkArr();
for(auto link:links){
  auto startNode = sdLinkLayer->GetLinkNodeByID(link->GetStartNodeID());
  auto endNode = sdLinkLayer->GetLinkNodeByID(link->GetEndNodeID());
}
```

3.3.1.3.8 通过 raw id 读取 Link: GetLinkByRawID

【功能说明】

根据 Link 的 raw id（原始 id），获取对应的 Link 对象。原始 id 的来源有：1. 腾讯地图的网络算路接口；2. link->GetRawID()。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
raw_link_id	请求参数，类型为 int64_t;	客户提供参数	是	

【输出结果】

字段	说明
ILinkConstPtr	link 对象。详细属性见 3.3.1.3.7 中的 ILinkConstPtr 表。

【异常说明】

无

【示例代码】

```
auto rawId = link->GetRawID();
auto link = sdLinkLayer->GetLinkByRawID(rawId);
```

3.3.1.3.9 读取 Link 属性 GetAttribute

【功能说明】

查询 llink 的通用属性，包含道路名、道路等级、方向等

【参数输入】

无

【输出结果】

字段	说明
LinkAttribute	link 的通用属性，包含 link 的道路名、道路等级、link 的方向等信息

link 的通用属性说明：

字段	说明
NameWithLanguage name	道路名称
std::string road_number	道路编号，比如 G7
RoadClassType road_class_type	link 的等级
RoadKindTypeVec road_kind_types	link 属性
FuncClassType func_class	功能类别
ConstructionType is_const	在建状态
RoadCondType road_cond	铺设状态
LinkDirection direction	link 方向
SpeedClassType speed_class	速度等级
AdminCode admin_code	行政区代码
uint8_t lane_num_sum	总车道数
uint8_t lane_num_s2e	S->E 方向的车道数
uint8_t lane_num_e2s	E->S 方向的车道数
uint8_t speed_limit_s2e	起点到终点方向的限速，单位：km/h
uint8_t speed_limit_e2s	终点到起点方向的限速，单位：km/h
std::vector<VehicleType> vehicle_types	车辆类型
RoadCompacity compacity	道路通行能力
bool is_special	特殊交通类型（**已经不维护**）
bool is_through;	是否可穿行
LinkTollType toll_type	道路收费类型
bool is_elevate	是否高架路（**已经不维护**）
bool u_flag	是否城市区域
bool is_cross_tile	是否跨 tile（**已经不维护**）
int width	link 宽度（**已经不维护**）
int length	link 长度
int16_t start_angle	起点角度[0,360),以正北方向顺时针计算
int16_t end_angle	终点角度[0,360),以正北方向顺时针计算
LinkSpeedSource speedsrc_ste	起点限速来源
LinkSpeedSource speedsrc_ets	终点限速来源
LinkDataSource data_source	数据来源（**已经不维护**）

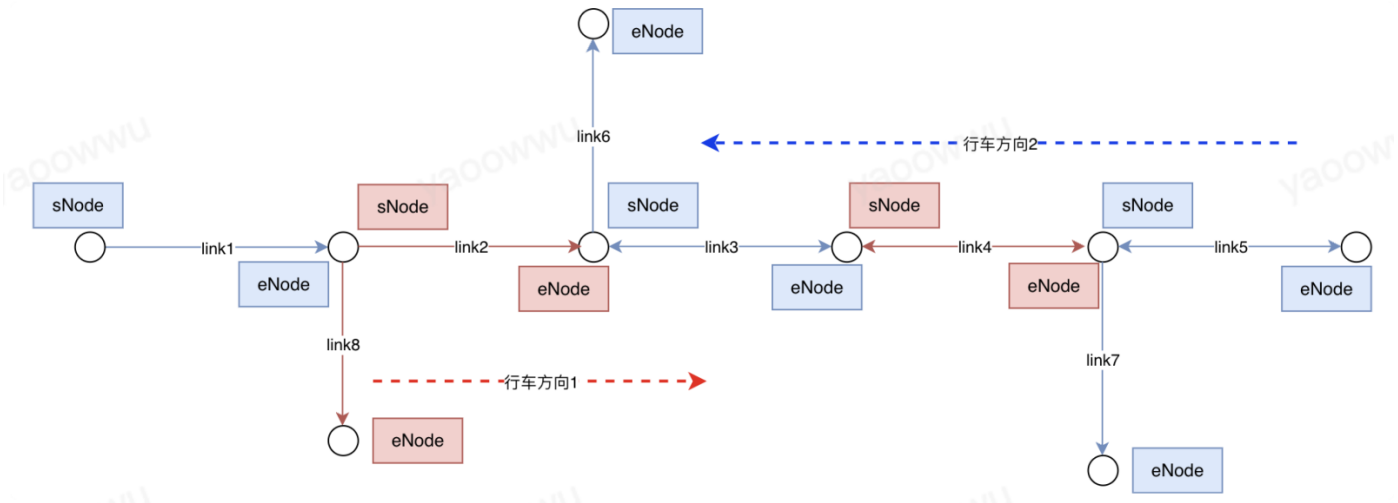
【异常说明】

无

【示例代码】

```
for(auto link:links){
  const auto &attr = link->GetAttribute();
  const auto & direction = attr.direction;
  const auto & roadName = attr.name;
  ...
}
```

3.3.1.3.10 读取 Link 拓扑



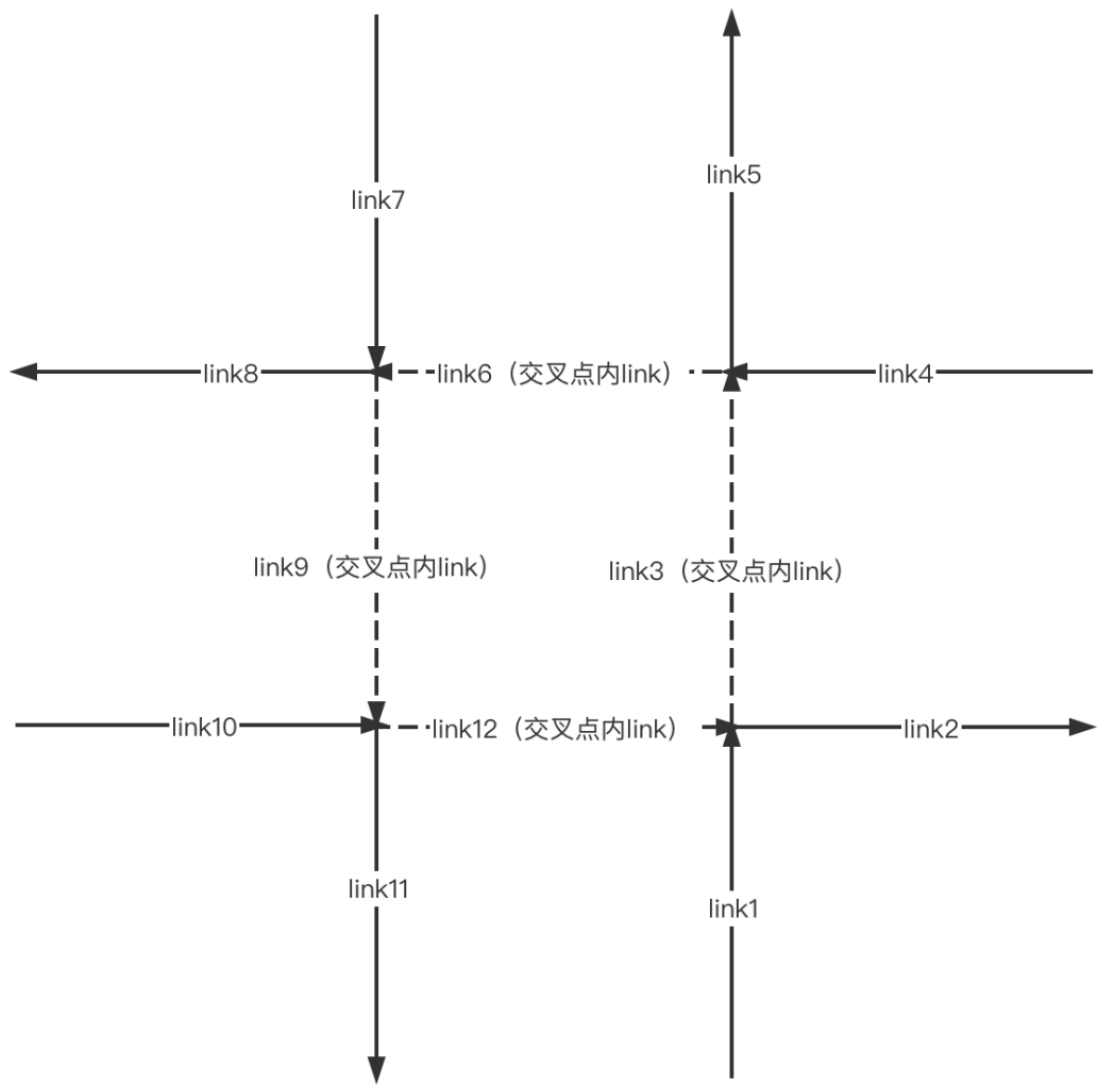
如上图，单向箭头表示正向 link，双向箭头表示双向 link，sNode 表示 link 的起点 node，eNode 表示 link 的终点 node。划线方向由 sNode 指向 eNode，顺向 link（kSTART_TO_END） 的实际可行驶方向和划线方向一致，双向 link（kBOTH_DIRECTION）由于可以双向同行，实际行驶方向可能和划线方向不一致；link 间的拓扑关系取决于 link 和 sNode/eNode 间的点线点关系；

读取 link 的拓扑关系时，在路口场景下，根据是否包含交叉点内 link，分为两类接口：

不包含交叉点内 link：GetPrevious/GetPreviouIDs，GetNext/GetNextIDs

包含交叉点内 link：GetPreviousWithinIntersection/GetPreviousIDsWithinIntersection GetNextWithinIntersection/GetNextIDsWithinIntersection

二者的区别解释如下：



从link1出发，调用link1->GetNext(nerd::api::LinkDirection::kSTART_TO_END)，返回的结果为link2、link5、link8、link11
调用link1->GetNextWithinIntersection(nerd::api::LinkDirection::kSTART_TO_END)，返回的结果为link2、link3

1、获取前继拓扑

（1）获取前继 link 列表：GetPrevious

【功能说明】

通过 link 对象查询当前 link 的前继 link 列表，列表中不包含交叉点内 link，并且**不包含跨 tile 的前继 link**:。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明：

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的前继 link 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
  auto prevLinks = link->GetPrevious(nerd: :api::LinkDirection::kSTART_T0_END);
}
```

（2）获取前继 linkID 列表：GetPreviouIDs

【功能说明】

通过 link 对象查询当前 link 的前继 link ID 列表，列表中不包含交叉点内 link ID，但是**包含跨 tile 的 link ID**。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明：

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的前继 link ID 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
  auto prevLinkIDs = link->GetPreviouIDs(nerd: :api::LinkDirection::kSTART_T0_END);
}
```

（3）获取前继 link 列表（包括交叉口内 link）：GetPreviousWithinIntersection

【功能说明】

通过 link 对象查询当前 link 的前继 link 列表，列表中包含交叉点内 link，但是**不包含跨 tile 的前继 link**:

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明：

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的前继 link 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
  auto prevLinks = link->GetPreviousWithinIntersection(nerd: :api::LinkDirection::kSTART_T0_END);
}
```

（4）获取前继 linkID 列表（包括交叉口内 linkID）：GetPreviousIDsWithinIntersection

【功能说明】

通过 link 对象查询当前 link 的前继 linkID 列表，列表中包含交叉点内 link ID，同时**包含了跨 tile 的前继 link ID**：

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明：

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的前继 link ID 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
  auto prevLinkIDs = link->GetPreviousIDsWithinIntersection(nerd: :api::LinkDirection::kSTART_T0_END);
}
```

2、 获取后继拓扑

（1）获取后继 link 列表：GetNext

【功能说明】

通过 link 对象查询当前 link 的后继 link 列表，列表中不包含交叉点内 link，并且**不包含跨 tile 的后继 link**:

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明:

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的后继 link 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
  auto nextLinks = link->GetNext(nerd::api::LinkDirection::kSTART_TO_END,false);//获取后继 link 列表
}
```

（2）获取后继 linkID 列表：GetNextIDs

【功能说明】

通过 link 对象查询当前 link 的后继 link ID 列表，列表中不包含交叉点内 link ID，但是**包含跨 tile 的 link ID**

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明:

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的后继 link ID 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
  auto nextLinkIDs = link->GetNextIDs(nerd::api::LinkDirection::kSTART_TO_END,false);//获取后继 link 列表
}
```

（3）获取后继 link 列表(包含路口内 link)：GetNextWithinIntersection

【功能说明】

通过 link 对象查询当前 link 的后继 link 列表，列表中包含交叉点内 link，但**不包含跨 tile 的后继 link**:

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明：

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的后继 link 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
 auto nextLinks = link->GetNextWithinIntersection(nerd::api::LinkDirection::kSTART_TO_END,false);//获取后继 link 列表
}
```

（4）获取后继 link 列表(包含路口内 link ID)：GetNextIDsWithinIntersection

【功能说明】

通过 link 对象查询当前 link 的后继 link ID 列表，列表中包含交叉点内 link ID，并且**包含跨 tile 的 link ID**。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
LinkDirection	查询拓扑的方向，详细见下方 link 方向说明	客户提供参数	是	
with_indoor	是否包含室内 Link	客户提供参数	否，可不传	空

link 方向说明：

字段	说明
kUNKNOWN	未定义，查询拓扑输入此参数无效
kBOTH_DIRECTION	双向（顺向、逆向），查询拓扑输入此参数无效
kSTART_TO_END	顺向
kEND_TO_START	逆向（不确定车辆在双向 link 上的行驶方向时，进行拓扑检查建议传入该参数多调用一次接口）

【输出结果】

字段	说明
std::vector<IDLinkConstPtr>	当前 link 的后继 link ID 列表

【异常说明】

无

【示例代码】

```
for(auto link:links){
 auto nextLinkIDs = link->GetNextIDsWithinIntersection(nerd::api::LinkDirection::kSTART_TO_END,false);//获取后继 link 列表
}
```

3.3.1.3.11 读取 Guidance GetGuidance

Guidance 类型（GuidanceType）说明如下：

字段	说明
kNone = 0	
kToll	收费站（**已经不维护**）

kGate	大门 V1.3 HA 引擎已经支持，SD 引擎可支持，未放开
kFork	分叉口提示（已经不维护）
kTrafficLight	交通信号灯（已经不维护）
kCamera	电子眼
kTollStation	收费站
kWarningSign	交通警示
kLaneMetaInfo	车信信息
kLsl	长实线
kBoard	方向看板 V1.3 最新引擎版本支持方向看板~~（已经不维护）~~
kPointSpeed	点限速
kTrafficLightCtrl	红绿灯控制 V1.3 最新引擎版本支持红绿灯控制 ~~（已经不维护）~~
kZLevelGroup	立交
kBranchPic	放大图（已经不维护）
kSlope	坡度 V1.3 最新引擎版本支持坡度~~（已经不维护）~~
kLaneInfo	车道信息（离线）（已经不维护）
kTurnModle	转向模型（离线）（已经不维护）

1 读取 link 上的车信信息

【功能说明】

获取当前 link 的车信信息（现实世界中路口的车道信息）

【参数输入】

参数	说明	参数提供方	是否必须	默认值
GuidanceType	guidance 类型，参考 Guidance 类型表，查询	客户提供参数	是	

	车信时类型为 kLaneMetaInfo			

【输出结果】

字段	说明
LaneInfoMeta	车信信息

车信使用：

车信结构由 3 部分组成：LaneInfoMeta, LaneTopo, LaneCond

LaneInfoMeta(车信信息 -> 继承于 IGuidanceInfo 诱导基类):

字段	类型	说明
lane_num	int	车道数
lane_infos	std::vector<LaneTrafficInfo>	车道通行信息(车道从左到右): 包含车道类型，是否公交车道，箭头信息（参考下方 lane_info 说明表）
lane_num_l	uint8_t	左附加车道数
lane_num_r	uint8_t	右附加车道数
lane_topos	std::vector<LaneTopo>	车道转向拓扑（参考下方 laneTopo 说明表）

lane_info: 车道通行信息，LaneTrafficInfo 结构中包含了 3 个字段：

字段	类型	说明
is_bus	bool	是否是公交车道（**已经不维护**）
lane_type	uint32_t	当前车道的车道类型，由多种车道类型枚举值通过与运算组成的一个 uint32_t 类型的数值(车道类型枚举参考 LaneExtensionType 结构)
arrow	uint32_t	当前车道的车道箭头信息，由多个箭头类型枚举值通过与运算组成的组成的一个 uint32_t 类型的数值(箭头类型枚举参考 LaneArrowType 结构)

LaneTopo(车信转向拓扑):

车信转向拓扑可以理解成从一条 link 到另一条 link 的一个路径

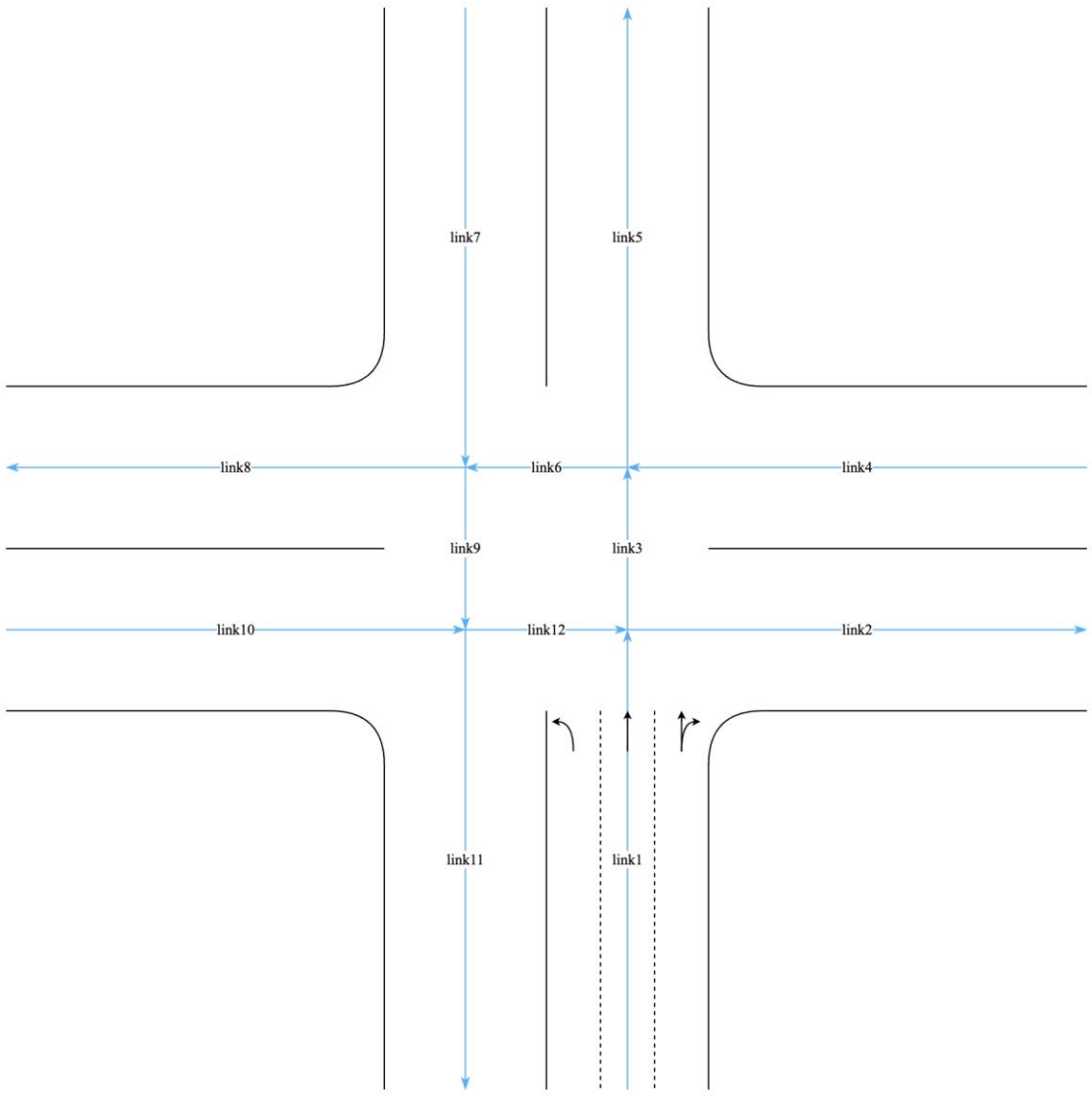
字段	类型	说明
lane_arrow	LaneArrowType	转向箭头

lane_flag	uint16_t	生效车道，16bit 从左到右表示车道是否生效
bus_flag	uint16_t	是否仅公交车生效，16bit 从左到右表示是否生效
out_link_ids	std::vector<LinkIDType>	退出线号
lane_conds	std::vector<LaneCond>	车信条件

LaneCond(车信条件):

字段	类型	说明
laneinfo_cond	uint16_t	生效车道，16bit 从左到右表示每个车道是否生效
direction	LinkDirection	车道方向
date	std::string	时间段
vehicles	std::vector<VehicleType>	限制车辆类型
special_times	std::vector<SpecialTimeType>	特殊时间类型

针对以上各字段的图示理解：



假如存在左图的场景, 对照车信LaneInfoMeta, LaneTopo结构，分析link1处的车信:
LaneInfoMeta:
- lane_num = 3
- lane_num_l = 0
- lane_num_r = 0
- lane_infos中的车道通行与车道数是一一对应
- lane_infos[0]: 从左到右第一个车道
  - arrow: 左转
  - is_bus: false
  - lane_type: 常规车道
- lane_infos[1]: 从左到右第二个车道
  - arrow: 直行
  - is_bus: false
  - lane_type: 常规车道
- lane_infos[2]: 从左到右第三个车道
  - arrow: 直行 + 右转
  - is_bus: false
  - lane_type: 常规车道

根据左图所示，link1可能的路径总共有3条:
1. link1->link2
2. link1->link3->link5
3. link1->link3->link6->link8

lane_topo的size为3。
lane_topos[0]:
- // 此处的箭头表示的就是link在当前路径下的转向信息
- lane_arrow: 左转
- lane_flag: 1000000000000000
- bus_flag: 0000000000000000
- out_links:[link3, link6, link8]
- lane_conds: 车信限制条件与每一个topo相关联

lane_topos[1]:
- // 此处的箭头表示的就是link在当前路径下的转向信息
- // 注意此处，直行的车道有两条，第2，3条车道均可直行
- lane_arrow: 直行
- lane_flag: 0110000000000000
- bus_flag: 0000000000000000
- out_links:[link3, link5]
- lane_conds: 车信限制条件与每一个topo相关联

lane_topos[2]:
- // 此处的箭头表示的就是link在当前路径下的转向信息
- lane_arrow: 右转
- lane_flag: 0010000000000000
- bus_flag: 0000000000000000
- out_links:[link2]
- lane_conds: 车信限制条件与每一个topo相关联

【异常说明】

无

【示例代码】

```
//车信信息
auto laneInfos = link->GetGuidance(nerd::api::GuidanceType::kLaneMetaInfo);
if(!laneInfos.empty()){
  std::cout << "laneInfo.size: " << laneInfos.size() << std::endl;
}
for (auto laneInfoPtr : laneInfos) {
  auto laneInfo = std::static_pointer_cast<const nerd::api::LaneInfoMeta>(laneInfoPtr);
  if (laneInfo == nullptr) {
    continue;
  }
  auto nodeid = laneInfo->GetNodeID();
  for (const auto &topo : laneInfo->lane_topos) {
    auto arrow = topo.lane_arrow;
    auto outLinkIds = topo.out_link_ids;
  }
}
```

2 读取 link 上的收费站信息

【功能说明】

获取当前 link 的收费站信息

【参数输入】

参数	说明	参数提供方	是否必须	默认值

GuidanceType	guidance 类型，参考 Guidance 类型表，查询车信时类型为 kTollStation	客户提供参数	是	

【输出结果】

字段	说明
TollStation	收费站信息

收费站字段说明：

字段	类型	说明
name	string	名称
direction	LinkDirection	作用方向
toll_means	uint8_t[]	通道收费方式，每个收费通道用 7 个 Bit 位组合表达，Bit 为 1 表示支持，为 0 表示不支持： Bit0：现金 Bit1：ETC Bit2：微信 Bit3：支付宝 Bit4：其他支付 Bit5：免费 Bit6：未调查 比如第 1 个通道支持 ETC，第 2 个通道支持现金/支付宝，第 3、4 通道支持现金/支付宝/其他支付可以表示为[2, 9,25,25]
passage_ways	uint8_t[]	通道类型，每个通道类型用 6 个 Bit 位组合表达，Bit 为 1 表示支持，为 0 表示不支持： Bit0：称重车道 Bit1：绿色车道 Bit2：通用车道 Bit3：预留 Bit4：预留 Bit5：预留 比如第一个收费口通道类型是通用车道，第二个收费口通道类型是称重+绿色通道可以表示为[4, 3]
gate_flag	TollGateType	收费站出入口标识，值为出口、入口、出入口
name_eng	string	英文名称，暂未使用（**已经不维护**）

separate_flag	TollSeparateType	分岔路标识，只针对经过收费站分叉道路进行标识、其他收费站设置为空
lane_group	uint8_t[]	通道方向分组,按照退出方向给车道进行分组，比如一共 7 车道，左边三个车道通往北京市区，右边四个车道通往北京机场，可表达为[3, 4], separate_flag 为 kExitMultiply 时有效
group_name	string[]	针对 lane group 中的分组车道，每组通往的不同方向名称，按照上面的示例表达为[北京市区,北京机场]，separate_flag 为 kExitMultiply 时有效

【异常说明】

无

【示例代码】

```
//收费站
auto guidancesToll = link->GetGuidance(nerd::api::GuidanceType::kTollStation);
if(!guidancesToll.empty()){
 std::cout << "guidancesToll.size: " << guidancesToll.size() << std::endl;
}
for (const auto &guidance : guidancesToll) {
 auto toll_station = std::static_pointer_cast<const nerd::api::TollStation>(guidance);
 if(toll_station == nullptr){
  continue;
 }
 auto name = toll_station->name;
 auto type = toll_station->GetType();
 auto toll_means = toll_station->toll_means;
 auto direction = toll_station->direction;
}
```

3 读取 link 上的点限速信息

【功能说明】

获取当前 link 的点限速信息

【参数输入】

参数	说明	参数提供方	是否必须	默认值
GuidanceType	guidance 类型，参考 Guidance 类型表，查询车信时类型为 kPointSpeed	客户提供参数	是	

【输出结果】

字段	说明
PointSpeed	点限速信息

点限速字段说明

字段	类型	说明
type	PointSpeedType	限速牌类型，最高限速、最低限速、解除限速等类型
speed_limit	uint8_t	限速值，单位：km/h
pre_dist	uint16_t	提前预告距离，单位：米

coordinate	Coordinate	坐标
direction	LinkDirection	关联道路的方向
affect_link_id	LinkIDType	作用道路 LinkId
affect_direction	LinkDirection	作用道路的方向
speed_scenes	PointSpeedScene[]	点限速场景，如隧道、匝道等
date	string	时间段限制，定义参考附录 6.2
special_times	SpecialTimeType[]	特殊时间类型
nav_coordinate	Coordinate	引导坐标，当点限速距离作用道路过远，记录引导坐标便于使用
location	PointSpeedLocation	相对位置，如相对作用道路的左侧、右侧等
vehicles	VehicleType[]	作用车辆类型

【异常说明】

无

【示例代码】

```
//点限速
auto guidancesSpeed = link->GetGuidance(nerd::api::GuidanceType::kPointSpeed);
if(!guidancesSpeed.empty()){
  std::cout << "guidancesSpeed.size: " << guidancesSpeed.size() << std::endl;
}
for (const auto &guidance : guidancesSpeed) {
  auto point_speed = std::static_pointer_cast<const nerd::api::PointSpeed>(guidance);
  if(point_speed == nullptr){
    continue;
  }
  auto speed_type = point_speed->type;
  auto position = point_speed->coordinate;
  auto speed_limit = point_speed->speed_limit;
  auto affect_link_id = point_speed->affect_link_id;
  auto special_times = point_speed->special_times;
  auto vehicles = point_speed->vehicles;
  auto speed_scenes = point_speed->speed_scenes;
}
```

4 读取 link 端点上的长实线信息

【功能说明】

获取当前 link 端点上的长实线信息（长实线是关联在 linkNode 上的）

【参数输入】

参数	说明	参数提供方	是否必须	默认值
GuidanceType	guidance 类型，参考 Guidance 类型表，查询车信时类型为 kLsl	客户提供参数	是	unknow

【输出结果】

字段	说明

LSL	长实线信息

长实线字段说明：

字段	类型	说明
seqnum	uint8_t	作用车道序号
lsl_type	LSLType	长实线类型，单实线、左虚右实、左实右虚等类型
pass_link_ids	LinkIDType[]	退出 link ID 列表，通过函数 GetPassLinkIDS()获取
lane_nums(2024 年 12 月 19 日新增)V1.3	uint8_t	总车道数
lsl_lanes(2024 年 12 月 19 日新增)V1.3	std::vector<uint8_t>	作用车道

【异常说明】

无

【示例代码】

```
//长实线
auto nodeId = link->GetStartNodeID();
auto node = sdLinkLayer->GetLinkNodeByID(nodeId);
if (node == nullptr) {
  continue;
}
auto guidancesLong = node->GetGuidance(nerd::api::GuidanceType::kLsl);
if(!guidancesLong.empty()){
  std::cout << "guidancesLong.size: " << guidancesLong.size() << std::endl;
}
for (const auto &guidance : guidancesLong) {
  auto lsl = std::static_pointer_cast<const nerd::api::LSL>(guidance);
  if(lsl == nullptr){
    continue;
  }
  auto type = lsl->lsl_type;
  auto exit_links = lsl->GetPassLinkIDS();
  auto lane_nums = lsl->lane_nums;
  auto lsl_lanes = lsl->lsl_lanes;
}
```

5 读取 link 上的交通警示信息

【功能说明】

获取当前 link 上的交通警示信息

【参数输入】

参数	说明	参数提供方	是否必须	默认值
GuidanceType	guidance 类型，参考 Guidance 类型表，查询车信时类型为 kWarningSign	客户提供参数	是	

【输出结果】

字段	说明
LinkWarningSign	交通警示信息

交通警示字段说明：

字段	类型	说明
trc_kind	TrafficSignType	警告牌类型，上陡坡、注意行人等警告标志
foretold_dist	uint16_t	提前预告距离，单位：米
control_dist	uint16_t	有效距离，单位：米
linkDirection	LinkDirection	作用方向
descript	string	文字说明，一般是文字，如“雨雾天气”、“急弯减速”（**暂不维护**）
coordinate	Coordinate	坐标

【异常说明】

无

【示例代码】

```
//交通警示
auto guidanceWarningSign = link->GetGuidance(nerd::api::GuidanceType::kWarningSign);
if(!guidanceWarningSign.empty()){
  std::cout << "guidanceWarningSign.size: " << guidanceWarningSign.size() << std::endl;
}
for(const auto &guidance : guidanceWarningSign){
  auto warningSign = std::static_pointer_cast<const nerd::api::LinkWarningSign>(guidance);
  auto trick_kind = warningSign->trc_kind;
  auto direction = warningSign->linkDirection;
  auto coord = warningSign->coordinate;
}
```

6 读取 link 上的 zlevel 信息

【功能说明】

获取当前 link 上的 zlevel 信息

【参数输入】

参数	说明	参数提供方	是否必须	默认值
GuidanceType	guidance 类型，参考 Guidance 类型表，查询车信时类型为 kZLevelGroup	客户提供参数	是	

【输出结果】

字段	说明
ZLevelGroup	zlevel 信息

zlevel 字段说明：

字段	类型	说明
links	std::vector<ZLinkLevel>	同一个立交关系组的 link
coordinate	Coordinate	坐标

【异常说明】

无

【示例代码】

```
//zlevel
auto guidanceZlevel = link->GetGuidance(nerd::api::GuidanceType::kZLevelGroup);
if(!guidanceZlevel.empty()){
  std::cout << "guidanceZlevel.size: " << guidanceZlevel.size() << std::endl;
}
for(const auto &guiance : guidanceZlevel){
  auto zlevel = std::static_pointer_cast<const nerd::api::ZLevelGroup>(guiance);
  auto links = zlevel->links;
  for(auto link : links){
    auto linkid = link.link_id;
    auto zValue = link.z;
  }
  auto coord = zlevel->coordinate;
}
```

7 读取 link 上的电子眼信息

【功能说明】

获取当前 link 上的电子眼信息

【参数输入】

参数	说明	参数提供方	是否必须	默认值
GuidanceType	guidance 类型，参考 Guidance 类型表，查询车信时类型为 kCamera	客户提供参数	是	

【输出结果】

字段	说明
Camera	电子眼信息

电子眼字段说明：

字段	类型	说明
type	CameraType	类型
group_id	uint64_t	分组编号，区间连续限速区域表示标识
coordinate	Coordinate	坐标
speed_limit	uint8_t	限速值
direction	LinkDirection	作用方向
position	ShootPosition	拍摄位置
start_date	std::string	开始作用时间
end_date	std::string	结束作用时间
vehicles	std::vector<VehicleType>	作用车辆类型
high_risk	bool	是否高危
scenes	std::vector<CameraScene>	场景
date	std::string	时间段

【异常说明】

无

【示例代码】

```
//电子眼
auto guidanceCamera = link->GetGuidance(nerd::api::GuidanceType::kCamera);
if(!guidanceCamera.empty()){
  std::cout << "guidanceCamera.size: " << guidanceCamera.size() << std::endl;
}
for(const auto &guidance : guidanceCamera){
  auto camera = std::static_pointer_cast<const nerd::api::Camera>(guidance);
  auto type = camera->type;
  auto coord = camera->coordinate;
  auto direction = camera->direction;
  auto shootPos = camera->position;
}
```

8 读取 link 上的基础坡度信息（2024 年 12 月 19 日新增）V1.3：

【功能说明】

获取当前 link 的基础坡度信息

【参数输入】

无

【输出结果】

使用方式：link->GetStartNodeSlopeO 和 link->GetStartNodeSlopeO，其返回值 LinkSlopeType 类型。

LinkSlopeType 定义如下：

```
/**
 * 坡度类型
 */
enum class LinkSlopeType : uint8_t {
  /**
   * 未调查
   */
  kNoSurvey = 0,
  /**
   * 上坡
   */
  kUpSlope,
  /**
   * 平坡
   */
  kFlatSlope,
  /**
   * 下坡
   */
  kDownSlope
};
```

【异常说明】

无

【示例代码】

```
nerd::api::LinkSlopeType startSlopeType = link->GetStartNodeSlope();
nerd::api::LinkSlopeType endSlopeType = link->GetEndNodeSlope();
```

9 读取 link 上的交通灯控制信息（2024 年 12 月 19 日新增）V1.3：

【功能说明】

获取当前 link 的交通灯控制信息

【参数输入】

无

【输出结果】

TrafficLightCtrlType 定义如下：

```
/**
 * 红绿灯控制类型
 */
enum class TrafficLightCtrlType : uint8_t {
  kNone = 0,
  /**
   * 掉头
   */
  kUTurn,
  /**
   * 借道左转
   */
  kBorrowLeft,
  /**
   * 右转
   */
  kTurnRight,
};
```

【异常说明】

无

【示例代码】

```
auto guidanceTrafficLightCtrl = link->GetGuidance(nerd::api::GuidanceType::kTrafficLightCtrl);
for(const auto &guidance:  guidanceTrafficLightCtrl){
  auto trafficLightCtrl = std::static_pointer_cast<const nerd::api::TrafficLightCtrl>(guidance);
  auto type = trafficLightCtrl->type;
}
```

3.3.1.3.12 根据经纬度坐标计算 tileId：nerd::api::utils::GetTileIdByGeoCoordinate

【功能说明】

根据经纬度坐标，获取其所属的地图瓦片的 tileId。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
coordinate	经纬度坐标，类型是 GeoCoordinate（x 是经度，y 是纬度）。	客户提供参数	是	
level	tile 切分等级；类型是 int32_t。（sd 的切分等级是 13，hd 的切分等级是 15）			

【输出结果】

字段	说明
TileIDType	经纬度坐标所在的地图瓦片的 tileId。

【异常说明】

无

【示例代码】

```
auto tileId = nerd::api::utils::GetTileIdByGeoCoordinate( nerd::api::GeoCoordinate(121.499721, 31.239846), 13);
```

3.3.1.3.14 获取数据版本 V1.3 新增

3.3.1.3.14.1 通过 data cube 对象获取版本号

【功能说明】

根据 data cube 对象获得其版本号

【参数输入】

【输出结果】

字段	说明
nerd::api::VersionIDRange	版本列表

VersionIDRange 说明：

```
/**
 * 瓦片数据的实际版本和全局版本
 */
class VersionIDRange {
 public:
  /**
   * 瓦片实际的数据版本（格式举例：1734520634）
   */
  VersionIDType data_ver_{0};
  /**
   * BuildingBlock 全局版本号，即 LCV(last confirmed version)。格式举例：1734520634
   */
  VersionIDType lcv_ver_{0};

  std::string toString() {
    std::stringstream ss;
    ss << "[" << data_ver_ << "," << lcv_ver_ << "]";
    return ss.str();
  }
};
```

【示例代码】

```
nerd::api::VersionIDRange versionRange = data_cube->GetVersionRange();
```

3.3.1.3.14.2 在线引擎：通过回调接口监听服务器最新版本号变化

【功能说明】

监听服务器最新的数据版本变化。

参数	说明	参数提供方	是否必须	默认值
online_status_callback	类型是 nerd::api::OnlineDataStatusChangeCallback	客户提供参数	否	

【输出结果】

字段	说明
nerd::api::MapDataBuildingBlockID bb_id	数据类型，例如 SD 数据，HD 数据等。类型是 MapDataBuildingBlockID，详见 MapDataBuildingBlockID 表。
uint32_t version_id	数据版本号

MapDataBuildingBlockID 的其他常见枚举值：

枚举值	说明
kROUTE	道路数据层（SD）
kLANE	高精数据层（HD）
……	……

【示例代码】

```
//该回调为检查到云端数据版本更新的回调
class OnlineDataStatusChangeCallbackImpl : public nerd::api::OnlineDataStatusChangeCallback{
 public:
  void OnNewData(nerd::api::MapDataBuildingBlockID bb_id, uint32_t version_id){
    if (bb_id == nerd::api::MapDataBuildingBlockID::kROUTE){
     //SD 数据版本发生变化
     std::cout << "SD Server data has been updated, bb_id = " << static_cast<int>(bb_id) << ",version:" << version_id << std::endl;
   }
  };
};

auto callback = std::make_shared<OnlineDataStatusChangeCallbackImpl>();//在线数据状态变化回调
  // 创建各个 building block 对应的引擎
  static auto create_engine =
     [&callback](nerd::api::MapDataBuildingBlockID build_block_id) -> std::unique_ptr<nerd::api::MapDataEngineApi> {
    nerd::api::APIConfig api_config;
    api_config.user_id = "building_block:" + std::to_string(static_cast<int32_t>(build_block_id));
    api_config.build_block_id = build_block_id;
    api_config.preference = nerd::api::DataPreference::kOnlineOnly;//在线模式
    api_config.online_status_callback = callback;
    return nerd::api::CreateEngine(api_config);
   };

 auto route_api = create_engine(nerd::api::MapDataBuildingBlockID::kROUTE);//创建获取 SD 数据的引擎实例
```

注意事项：在本地有缓存的情况下，首次启动进程创建引擎后，api_config.online_status_callback 会立刻触发回调，此时并不代表有了新版本的数据。

3.3.1.3.15 获取车道数变化点 V1.3 新增

【功能说明】

可用此方式获取车道数变化点信息。

注意：如果需要车道数量变化点信息，需要先确保对应渠道号有此数据权限，再打开对应的请求参数代码。

车道数量变化点信息非标准 SD 数据，需要额外配置。

【参数输入】

如果需要车道数变化点信息，需要给 GetMapDataByRectParam 或 GetMapDataBatchParam 的 feature_set_map 参数赋值。

参数	说明	参数提供方	是否必须	默认值
feature_set_map	std::map<DataFeatureType, DataFeatureSet>类型	客户提供参数	如果需要车道数变化点信息，则为必需参数。	

【输出结果】

LinkExtAttribute 属性说明：

字段	说明
std::vector<LaneNumChangePointInfo> lane_change_point_info_list	车道数变化点信息

LaneNumChangePointInfo 属性说明：

字段	说明
bool is_from_start_to_end	表示是否从起点到终点
uint8_t previous_lane_number	前继车道数量
uint8_t current_lane_number	当前车道数量
Coordinate position;	车道变化位置点

【示例代码】

```
nerd::api::GetMapDataBatchParam route_batch_param;
route_batch_param.tile_id_list.insert(route_tile_id);

//修改 feature_set_map 可设置要拉取的 link 附加属性数据: 车道数量变化点信息(对应 LINK_ATTRIBUTE_TYPE_ROAD_LANE_NUMBER_CHANGE)

route_batch_param.feature_set_map  = {
   {nerd::api::DataFeatureType::kRouteLinkAttribute,
    {nerd::api::LINK_ATTRIBUTE_TYPE_ROAD_LANE_NUMBER_CHANGE}
   }
};

auto route_tile_callback = std::make_shared<TileCallback>();
 route_api->AsyncGetDataByTileId(route_batch_param, route_tile_callback);
//等待 tile 结果
route_tile_callback->Wait();

...

//得到 link 对象后
const auto& linkExtAttr = link->GetExtAttribute();
std::cout <<"GetExtAttribute: "
        << "车道数量变化点信息数量:" << linkExtAttr.lane_change_point_info_list.size()<<std::endl;

if (linkExtAttr.lane_change_point_info_list.size() > 0) {
 std::string info_str = "[";
 for (std::size_t i = 0; i < linkExtAttr.lane_change_point_info_list.size(); ++i) {
   std::string item_str ="{";
   auto& item = linkExtAttr.lane_change_point_info_list[i];
   item_str += "是否从起点到终点:" + std::string(item.is_from_start_to_end?"是":"否") + ",";
   item_str += "前继车道数量:" + std::to_string(item.previous_lane_number) + ",";
   item_str += "当前车道数量:" + std::to_string(item.current_lane_number) + ",";
   item_str += "position:" + item.position.ToGeo().ToString() + ",";
   item_str+="}";
   info_str = info_str + item_str+",";
  }
  info_str+="]";
  std::cout << "  车道数量变化点信息: " << info_str << std::endl;
 }
```

3.3.1.3.16 用指定的数据版本号请求数据 （V1.3 新增）

【功能说明】

可用此功能请求固定版本的服务器数据（服务器支持双版本号数据，只能设置这两个版本号的其中一个）。

【参数输入】

如果需要车道数变化点信息，需要给 GetMapDataByRectParam 或 GetMapDataBatchParam 的 feature_set_map 参数赋值。

参数	说明	参数提供方	是否必须	默认值
MapDataBuildingBlockID bb_id	nerd::api::MapDataBuildingBlockID 类型，例如 SD 数据，HD 数据等，详见 MapDataBuildingBlockID 表。	客户提供参数	是	
std::string format_version	字符串类型，表示数据协议版本号，目前的数据协议版本号是 6.0。	客户提供参数	是	
nerd::api:: VersionIDType version_id	表示要设置的版本号，例如 1735812001	客户提供参数	是	

【输出结果】

无

【示例代码】

```
//将 SD 数据的版本号固定为 1735812001
nerd::api::SetDataRequestVersion(nerd::api::MapDataBuildingBlockID::kROUTE, "6.0", 1735812001);

//请求数据
nerd::api::GeoCoordinate point{121.499721, 31.239846};
//以某一点为中心，半径 500 米，构建一个矩形框
nerd::api::Rect rect = nerd::api::utils::BuildRectByLength(point,500);

nerd::api::GetMapDataByRectParam rect_param;
rect_param.level = nerd::api::MapLevel::kLEVEL_13;//获取 SD 数据(地图缩放层级 13 层)
rect_param.rect = rect;
auto rect_callback = std::make_shared<RectCallback>();
route_api->AsyncGetDataByRect(rect_param, rect_callback);
// 等待结果
rect_callback->Wait();

auto result = rect_callback->GetResult();
if(result->data_cube != nullptr){
   auto sdLinkLayer = result->data_cube->GetSDLinkLayer();
   getSDMapElement(sdLinkLayer);
}
```

3.3.1.4 数据引擎 HD Air 接口介绍

3.3.1.4.1 停止线信息：

1. 停止线的通用属性说明

StopLine 的通用属性说明：

字段	说明

StopLineIDType GetID()	获取该元素对应的 ID
StopLineType GetStopLineType()	获取停止线的类型。StopLineType 详见下表。
Coordinate GetLocation()	获取停止线的位置坐标
NodeIDType GetRelatedNodeID()	获取关联的 NodeID 信息
std::vector<LaneIDType> GetRelatedLaneIDs()	获取关联的 LaneID 信息
std::vector<IntersectionRoadID> GetRelatedIntersectionRoadIDs()	获取关联的 IntersectionRoadID 信息

StopLineType 的定义：

```
/**
* HDAirToB 定义停止线类型枚举
*/
enum class StopLineType : uint8_t {
 SLT_Unknown = 0,                // 未分类
 SLT_StopLine,                   // 停止线
 SLT_YieldLine,                  // 停车让行线
 SLT_DecelerationLine,           // 减速让行线
 SLT_VirtualStopLine,            // 虚拟停止线
 SLT_SlowLine,                   // 慢行线
 SLT_LeftTurnWaitingAreaStopLine,      // 左转待转区停止线
 SLT_StraightWaitingAreaStopLine,      // 直行待行区停止线
 SLT_Other,                      // 其他
 SLT_TideLaneStopLine,           // 潮汐车道停止线
 SLT_ReverseStopLine,            // 逆向停止线
 SLT_OtherWaitingAreaStopLine,         // 其他待转区停止线
 SLT_RightTurnWaitingAreaStopLine,     // 右转待转区停止线
 SLT_TurnRoundWaitingAreaStopLine      // 调头待转区停止线
};
```

2. 根据 LinkId 获取停止线 Id 列表: GetStopLineIDsByLinkId

【功能说明】

根据 LinkID 获得其所属的停止线列表。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
linkId	类型是 LinkIDType	客户提供参数	是	

【输出结果】

字段	说明
std::vector<nerd::api::hdair::StopLineIDType>	停止线 ID 列表

【示例代码】

```
if(data_cube != nullptr) {
  std::unique_ptr<IDataCube> data_cube = result->data_cube;
  auto sdLaneLayer = result->data_cube->GetSDLinkLayer();
  if(sdLaneLayer!=nullptr){
    ...
    auto stopLineIds = sdLinkLayer->GetStopLineIDsByLinkId(link->GetID());
  }
}
```

3. 获得所有的停止线 ID 列表：GetStopLineIDs

【功能说明】

获得 tile 内所有的停止线 ID 列表。

【参数输入】

无

【输出结果】

字段	说明
std::vector<nerd::api::hdair::StopLineIDType>	停止线 ID 列表

【示例代码】

```
if(data_cube != nullptr) {
  std::unique_ptr<IDataCube> data_cube = result->data_cube;
  auto sdLaneLayer = result->data_cube->GetSDLinkLayer();
  if(sdLaneLayer!=nullptr){
    ...
    auto stopLineIds = sdLinkLayer->GetStopLineIDs();
  }
}
```

4. 根据停止线 Id 获取停止线对象: GetStopLineById

【功能说明】

根据停止线 Id 获得停止线对象。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
stopLineId	类型是 nerd::api::hdair::StopLineIDType	客户提供参数	是	

【输出结果】

字段	说明
std::shared_ptr<nerd::api::hdair::IStopLine>	停止线对象

【示例代码】

```
auto stopLine = sdLinkLayer->GetStopLineById(stopLineId);
```

3.3.1.4.2 变化点 FeaturePoint 信息：

1. **FeaturePoint 通用属性说明**

FeaturePoint 的通用属性说明：

字段	说明
FeaturePointIDType GetID()	获取该元素对应的 ID, 通过索引动态生成;
std::vector<FeaturePointType>& GetTypes()	获取变化点类型类别。FeaturePointType 详见下表。
LinkDirection GetDirection()	获取方向(双向/正向逆向信息)
Coordinate GetLocation()	获取 FeaturePoint 位置坐标
float GetProjectPercent()	获取投影比例, 值域: [0.0, 1.0](投影到 link 的比例，从 link 起点开始计算，如 45%位置记录为 0.45)
std::vector<LaneIDType>& GetLaneIDs()	获取该点包含的车道 ID 列表
std::vector<std::shared_ptr<ILane>>& GetLanes()	获取该点包含的车道信息
std::vector<FeaturePointIDType> GetPreviousIDs()	获取该 FeaturePoint 关联的前序 FeaturePointID 列表
std::vector<FeaturePointIDType> GetNextIDs()	获取该 FeaturePoint 关联的后继 FeaturePointID 列表

FeaturePointType 定义：

```
/**
 * HDAirToB 定义特征点变化类型枚举
 */
enum class FeaturePointType : uint8_t {
  LaneTypeChange,                    // 车道类型变化
  LaneCountChange,                   // 车道数变化
  DataBoundaryStart,                 // 数据边界开始
  DataBoundaryEnd,                   // 数据边界结束
  LaneContinuityPoint,               // 车道连续点
  ExchangeAreaStart,                 // 交换区起点
  ExchangeAreaEnd,                   // 交换区终点
  RegularIntersectionEntrance,       // 普通路口进入点
  RegularIntersectionExit,           // 普通路口退出点
  UTurnIntersectionEntrance,         // 掉头路口进入点
  UTurnIntersectionExit,             // 掉头路口退出点
  WideLaneEntrance,                  // 宽车道进入点
  WideLaneExit,                      // 宽车道退出点
  TollBoothEntrance,                 // 收费站进入点
  TollBoothExit,                     // 收费站退出点
  ExchangeAreaInteriorPoint,         // 交换区内点
  WideLaneInteriorPoint,             // 宽车道内点
  TollBoothInteriorPoint,            // 收费站内点
  MarkingLineChangePoint             // 标线变化点
};
```

2. 根据 LinkId 获取 FeaturePointID 列表 : GetFeaturePointIDsByLinkId

【功能说明】

根据 LinkID 获得其所属的 FeaturePoint 列表。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
linkId	类型是 LinkIDType	客户提供参数	是	

【输出结果】

字段	说明
std::vector<nerd::api::hdair::FeaturePointIDType>	FeaturePoint ID 列表

【示例代码】

```
if(data_cube != nullptr) {
  std::unique_ptr<IDataCube> data_cube = result->data_cube;
  auto sdLaneLayer = result->data_cube->GetSDLinkLayer();
  if(sdLaneLayer!=nullptr){
    ...
    auto featurePointIDs = sdLinkLayer->GetFeaturePointIDsByLinkId(link->GetID());
  }
}
```

3. 获得所有 FeaturePointID 列表：GetFeaturePointIDs

【功能说明】

返回范围内所有的 FeaturePointID 列表。

【参数输入】

无

【输出结果】

字段	说明
std::vector<nerd::api::hdair::FeaturePointIDType>	FeaturePoint ID 列表

【示例代码】

```
if(data_cube != nullptr) {
  std::unique_ptr<IDataCube> data_cube = result->data_cube;
  auto sdLaneLayer = result->data_cube->GetSDLinkLayer();
  if(sdLaneLayer!=nullptr){
    ...
    auto featurePointIDs = sdLinkLayer->GetFeaturePointIDs();
  }
}
```

4. 根据 FeaturePointID 获取 FeaturePoint 对象：GetFeaturePointById

【功能说明】

根据 FeaturePointID 获取 FeaturePoint 对象。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
featurePointId	类型是 nerd::api::hdair::FeaturePointIDType	客户提供参数	是	

【输出结果】

字段	说明
std::shared_ptr<nerd::api::hdair::IFeaturePoint>	FeaturePoint 对象

【示例代码】

```
    if(data_cube != nullptr) {
      std::unique_ptr<IDataCube> data_cube = result->data_cube;
      auto sdLaneLayer = result->data_cube->GetSDLinkLayer();
      if(sdLaneLayer!=nullptr){
        ...
        auto featurePointIDs = sdLinkLayer->GetFeaturePointIDsByLinkId(link->GetID());
        for(auto &pointId : featurePointIDs){
         auto point = sdLinkLayer->GetFeaturePointById(pointId);
        }
      }
    }
```

3.3.1.4.3 斑马线：

1. 斑马线(IZebra)的通用属性说明

IZebra 的接口/属性说明：

字段	说明
FeatureIDType GetID()	获取该元素对应的 ID
LaneGroupDirection GetDirection()	获取方向(双向/正向逆向信息)
Coordinate GetLocation()	获取位置坐标
NodeIDType GetRelatedNodeID()	获取关联的 NodeId 信息
LinkIDType GetRelatedLinkID()	获取所在 Link 的 LinkId

2. 根据 link 获取斑马线 ID：GetZebraIDsByLinkId

【功能说明】

返回指定 LinkID 的 ZebraID 列表。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
linkId	类型是 LinkIDType	客户提供参数	是	

【输出结果】

字段	说明

std::vector<hdair::FeaturePointIDType>	ZebraID 列表

【示例代码】

```
// 通过 link id 获取斑马线 id 列表
auto zebraIdsInLink = sdLinkLayer->GetZebraIDsByLinkId(link->GetID());
for (auto zebraId : zebraIdsInLink) {
  auto zebra = sdLinkLayer->GetZebraById(zebraId);
  std::cout << "linkId=" << link->GetID().ToString() << ", zebraId=" << zebra->GetID().ToString() << std::endl;
}
```

3. **获取所有斑马线 ID：GetZebraIDs**

【功能说明】

获取范围内所有的斑马线 id 列表

【参数输入】

【输出结果】

字段	说明
std::vector<hdair::FeaturePointIDType>	ZebraID 列表

【示例代码】

```
// 获取所有的斑马线 id 列表
auto zebraIdsInLinkLayer = sdLinkLayer->GetZebraIDs();
for (auto zebraId : zebraIdsInLinkLayer) {
  auto zebra = sdLinkLayer->GetZebraById(zebraId);
  std::cout << "zebraId=" << zebra->GetID().ToString() << std::endl;
}
```

4. **根据斑马线 ID 获取斑马线对象：GetZebraById**

【功能说明】

根据 ZebraID 查询 Zebra 对象信息。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
fId	类型是 hdair::FeaturePointIDType	客户提供参数	是	

【输出结果】

字段	说明
std::shared_ptr<hdair::IZebra>	Zebra 对象。空则代表没有找到，需要到对应 TileID 的瓦片中找

【示例代码】

```
for (auto zebraId : zebraIds) {
  auto zebra = sdLinkLayer->GetZebraById(zebraId);
  std::cout << "zebraId=" << zebra->GetID().ToString() << std::endl;

  // 获取斑马线相关属性
  auto zebraGetDirection = zebra->GetDirection();
  auto zebraLocation = zebra->GetLocation();
  auto zebraRelatedLinkID = zebra->GetRelatedLinkID();
  auto zebraRelatedNodeID = zebra->GetRelatedNodeID();
  std::cout << "zebraId=" << zebra->GetID().ToString()
        << ", zebraGetDirection=" << (int)zebraGetDirection
        << ", zebraLocation=" << zebraLocation
        << ", zebraRelatedLinkID=" << zebraRelatedLinkID.ToString()
        << ", zebraRelatedNodeID=" << zebraRelatedNodeID.ToString() << std::endl;

}
```

3.3.1.4.4 路口内道路：

1. **路口内道路说明**

IIntersectionRoad 的通用属性说明：

字段	说明
IntersectionRoadID GetRoadID()	获取 NodeID
LinkDirection GetDirection()	获取方向(双向/正向逆向信息)
CrossTurnType GetCrossTurnType()	获取路口的转向信息
bool GetTrafficLightFlag()	获取红绿灯标识，有红绿灯返回 true
std::shared_ptr<std::vector<Coordinate>> GetGeometry()	获得 Road 几何形状
IntersectionRoadNodeID GetStartNodeID()	获得当前 Road 起点的 NodeID
IntersectionRoadNodeID GetEndNodeID()	获得当前 Road 终点的 NodeID
FeaturePointIDType GetStartFeaturePointID()	获得当前 Road 起点的 FeaturePointID
FeaturePointIDType GetEndFeaturePointID()	获得当前 Road 终点的 FeaturePointID
LinkIDType GetRelatedLinkID()	获取当前 Road 的 LinkID
std::vector<IntersectionRoadID> GetNextIDs(LinkDirection direction)	获取 IntersectionRoad 的下游 IntersectionRoadID
std::vector<IntersectionRoadID> GetPreviousIDs(LinkDirection direction)	获取 IntersectionRoad 的上游 IntersectionRoadID

CrossTurnType 类型定义：

```
/**
 * 路口车道转向信息
 */
enum class CrossTurnType : uint8_t {
  NONE = 0,        /** 不应用 */
  STRAIGHT = 1,     /** 直行 */
  LEFT = 2,        /** 左转 */
  RIGHT = 3,        /** 右转 */
  LEFT_TURN = 4,     /** 左掉头 */
  RIGHT_TURN = 5     /** 左掉头 */
};
```

2. 获取 sdLinkLayer 内所有的路口内虚拟道路：GetIntersectionRoadArr

【功能说明】

获取范围内所有的路口内虚拟道路。

【参数输入】

【输出结果】

字段	说明
std::vector<std::shared_ptr<hdair::IIntersectionRoad>>	路口内虚拟道路列表

【示例代码】

```
// 获取 sdLinkLayer 内所有的路口内道路
auto intersectionRoads = sdLinkLayer->GetIntersectionRoadArr();
for (const auto &intersectionRoad : intersectionRoads) {
  ...
}
```

3. 通过路口内道路 Node ID 获取路口内道路 Node 对象:

【功能说明】

根据 xxxxx 获得其所属的 xxxxx。

【参数输入】

参数	说明	参数提供方	是否必须	默认值
xxxxxx	类型是 xxxxx	客户提供参数	是	

【输出结果】

字段	说明
xxxx	xxxxx

【示例代码】

4. 通过 LinkId 获取关联的路口内道路对象：GetIntersectionRoadIDsByLinkId

【功能说明】

返回指定 LinkID 关联的 IntersectionRoadID 列表

【参数输入】

参数	说明	参数提供方	是否必须	默认值
linkId	类型是 LinkIDType	客户提供参数	是	

【输出结果】

字段	说明
std::vector<hdair::IntersectionRoadID>	IntersectionRoadID 列表

【示例代码】

```
// 通过 link id 获取 LinkID 关联的 IntersectionRoadID 列表
auto intersectionRoadIDs = sdLinkLayer->GetIntersectionRoadIDsByLinkId(link->GetID());
```

5. 通过 FeaturePointId 获取关联的路口内道路对象：GetIntersectionRoadIDsByFpId

【功能说明】

返回指定 FeaturePointID 关联的 IntersectionRoadID 列表

【参数输入】

参数	说明	参数提供方	是否必须	默认值
fpId	类型是 hdair::FeaturePointIDType	客户提供参数	是	

【输出结果】

字段	说明
std::vector<hdair::IntersectionRoadID>	IntersectionRoadID 列表

【示例代码】

```
// 通过 featurePoint Id 获取 FeaturePointID 关联的 IntersectionRoadID 列表
auto intersectionRoadIds = sdLinkLayer->GetIntersectionRoadIDsByFpId(featurePointId);
```

6. 通过 路口内道路 Id 获取路口内道路对象：GetIntersectionRoadByID

【功能说明】

根据路口虚拟道路 ID 查询指定的道路数据

【参数输入】

参数	说明	参数提供方	是否必须	默认值
roadId	类型是 hdair::IntersectionRoadID	客户提供参数	是	

【输出结果】

字段	说明
std::shared_ptr<hdair::IIntersectionRoad>	路口内道路对象

【示例代码】

```
// 通过路口内道路 id 获取路口内道路对象
auto intersectionRoad = sdLinkLayer->GetIntersectionRoadByID(intersectionRoadId);
```

3.3.2 离线数据管理模块（data-manager）

3.3.2.1 离线数据管理功能介绍

离线数据管理引擎是一个专门设计用于高效管理本地底图数据和路线数据的系统。其主要功能包括按城市（地级市）进行数据的下载、更新和删除，旨在为用户提供便捷、灵活的数据管理体验。

数据下载：

按城市下载：用户可以根据需要选择特定的地级市进行底图数据和路线数据的下载。系统会提供可用城市的列表，用户可以轻松选择所需的城

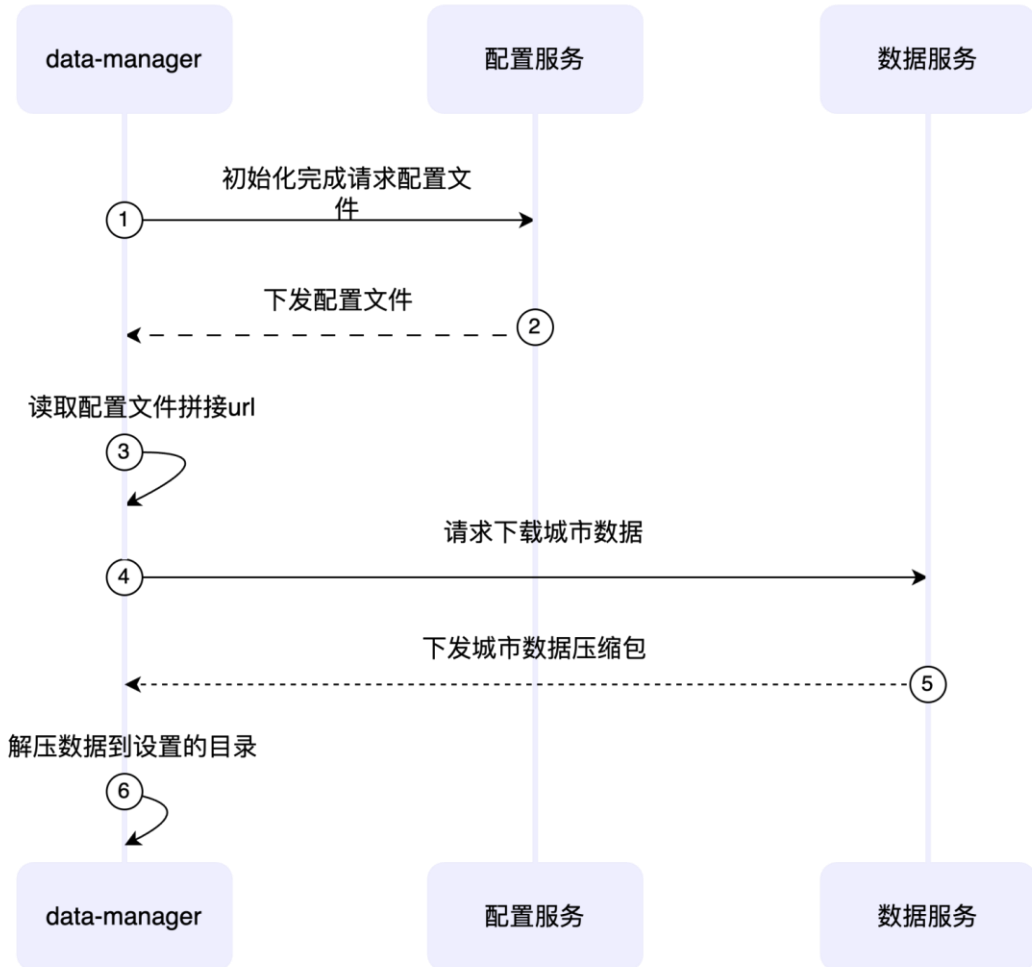
市。
多任务下载：为了提高下载效率，离线数据管理引擎支持多线程下载，能够同时下载多个城市的数据，缩短整体下载时间。

数据更新：
定期更新：离线数据管理引擎定期检查可用的更新，确保用户的底图数据和路线数据始终保持最新状态。用户可以设置自动更新的频率。
更新通知：在有新版本可用时，离线数据管理引擎收到查询通知时，会向用户发送通知，告知其可进行更新，确保用户及时获取最新数据。

数据删除：
按城市删除：用户可以选择删除特定城市的底图数据和路线数据，系统会提供确认提示，以防止误操作。
批量删除：支持批量删除功能，用户可以一次性删除多个城市的数据，方便管理存储空间。
清理无效数据：离线数据管理引擎会在开机时自动识别并清理无效或过期的数据，确保系统的高效运行。

数据管理与监控：
日志记录：离线数据管理引擎会记录所有操作的日志，包括下载、更新和删除操作，便于后续审计和问题排查。
更新进度通知：可以实时通知当前数据下载更新的状态。

通过以上功能，离线数据管理引擎（DM）为用户提供了一个全面、高效且安全的数据管理解决方案，使用户能够轻松管理本地的底图数据和路线数据，满足不同场景下的需求。



注：全国数据量 7.26G(1030 版本数据)

1. **初始化完成请求配置文件**：
   a. 离线数据管理引擎启动时，向配置服务请求配置文件，查询是否有新的更新。
   b. 确保配置文件格式正确，并能够被后续步骤解析。
2. **配置服务下发配置文件**：
   a. 如果有新的配置文件，配置服务下发最新的配置文件。
   b. 配置文件应包含可用城市的列表、每个城市的数据版本、下载地址、更新说明等信息。
   c. 处理配置服务的响应，确保接收到的配置文件完整且有效。
3. **解析配置文件**：
   a. 离线数据管理引擎解析下发的配置文件，提取出数据服务的下载地址、城市代码、数据版本等关键信息。
   b. 验证解析出的信息是否符合预期，确保后续下载请求的准确性。
4. **业务根据 city code 发起请求下载离线数据后，离线数据管理引擎向数据服务请求下载**：

a. 根据用户选择的城市代码，离线数据管理引擎构建下载请求，包含必要的参数（如城市代码、渠道号等）。
b. 向数据服务发送下载请求，确保请求格式正确，并处理可能的网络异常。

5. **数据服务收到请求后，下发该城市的数据压缩包：**

a. 数据服务接收到请求后，验证请求的合法性，并准备相应城市的数据压缩包。
b. 数据服务将压缩包发送给离线数据管理引擎，并提供下载进度信息。

6. **离线数据管理引擎解压数据到设置的目录：**

a. 离线数据管理引擎接收数据压缩包后，首先进行完整性校验（如 MD5 校验），确保数据未损坏。
b. 解压数据到预设的目录，确保解压后的文件结构符合预期。
c. 更新本地数据索引，以便新数据能够被正确识别和使用。

【功能输入信息】

1、初始化请求配置文件：

配置项	作用说明	参数提供方	是否必须	默认值
channel	渠道号，由腾讯项目提供，一般一个车型一个渠道号，没有配置正确的渠道号无法获取异源匹配和诱导能力，服务根据不同的渠道号下发不同的定制信息	腾讯提供参数	是	unkonw
imei	设备唯一标识，由 SDK 调用方提供，主要用于设备的鉴权和调查问题是日志信息查询	客户提供参数	是	“”
log_path	日志存放路径	客户提供参数	否	当前日志存放路径默认为：./log
data_path	数据存放路径	客户提供参数	否	./data
tca_log_level	日志级别,可以为 VERBOSE = -2, 详细的日志信息 DEBUG = -1, 调试日志 INFO = 0, 一般信息 WARNING = 1, 警告日志 ERROR = 2, 错误 FATAL = 3, 致命错误 NONE = 4, 不输出日志	客户提供参数	否	日志级别的默认值为 INFO = 0;
log_name_prefix	日志文件名前缀,如果为空,则使用默认配置，日志文件名格式: tencent_tca_20240912_202517_977.log.gz	客户提供参数	否	当前日志文件名前缀为：tencent_tca_
log_storage_time_limit	日志存储时间限制,单位为天,如果小于等于 0 表示使用默认配置	客户提供参数	否	日志存储时间默认值为：30 天
log_storage_total_size_limit	日志存储大小的最大限制，单位为 Byte，如果小于等于 0 表示使用默认配置	客户提供参数	否	日志存储大小默认为 200MB
log_storage_single_size_limit	单个日志文件大小限制，单位为 Byte，如果小于等于 0 表示使用默认配置	客户提供参数	否	日志文件大小默认为 5MB

【接口信息说明】

接口名称	参数	参数提供方	接口能力说明
Create	[in] const std::Setting& setting: 初始化所需配置	客户提供参数	静态方法， 用于创建数据管理对象的实例，同步接

	[in] std::shared_ptr <mapbase::HttpInterface> http： 实现 http_interface 接口的对象		口。
Init	\	客户提供参数	初始化方法，**调用其他方法前必须调用此方法** 通过 OnInitFinished()回调完成，如果 SetDownloadListener 在初始化完成之后调用，收不到回调，同步接口。
HasInited	\	\	初始化是否完成，同步接口。
CheckNewVer	[in] std::weak_ptr <DataUpdateListener> listener: 数据更新监听器	客户提供参数	检查是否有数据更新，成功返回 true，检查完毕回调最新的省市列表 OnNewDataTable。
GetDataTable	[out] std::vector <DownloadDataItem> & items: 城市列表	腾讯提供参数	获取城市列表，成功返回 true， 与 OnNewDataTable 返回结果相同， HasInited 返回 true 调用才能生效果，同步接口。
Set DownloadListener	[in] std::weak_ptr <DownloadListener> listener：下载监听器	客户提供参数	设置下载监听器，用于监听 下载、暂停、恢复、取消和删除等数据内容和状态的变化， 同步接口。
RequestDownload	[in] int32_t download_id: 下载城市的行政区号	客户提供参数	下载城市数据，如果已经下载且有更新则更新数据， HasInited 返回 true 才能调用， 否则返回 false，对于省份 ID（对应省的行政区号）下载， 依次回调其包含的城市数据 下载的数据状态变化：下载排队 -> 下载中 -> 已下载（失败为下载暂停或未下载） 更新的数据状态变化：更新排队 -> 更新中 -> 已下载（失败为更新暂停或已下载有更新）
CheckRegionsData	\	\	如果需要获取行政区划信息，请先下载更新全国行政区文件 成功时回调 DemoDownloadListener::OnDownloadFinished, 失败时回调 DemoDownloadListener::OnErrorMessage, 回调时 download_id 为 0。
PauseDownload	[in] int32_t download_id： 下载城市的行政区号	客户提供参数	暂停下载或更新，成功返回 true， HasInited 返回 true 才能调用。
ResumeDownload	[in] int32_t download_id： 下载城市的行政区号	客户提供参数	恢复下载或更新，成功返回 true， HasInited 返回 true 才能调用。
Remove DownLoad	[in] int32_t download_id： 下载城市的行政区号	客户提供参数	删除下载数据，成功返回 true，HasInited 返回 true 才能调用，回调

			DownloadListener::OnDownloadStatusChanged。
Cancel Downloading	**[in] int download_id：** 下载城市的行政区号	客户提供参数	取消所有更新和下载，成功返回 true，HasInited 返回 true 才能调用。
UpdateAll Downloaded	\	\	更新所有已下载数据，成功返回 true，HasInited 返回 true 才能调用。
PauseAllDownload	\	\	暂停所有下载，成功返回 true，HasInited 返回 true 才能调用。
Resume AllDownload	\	\	恢复所有下载，成功返回 true，HasInited 返回 true 才能调用。
CancelAll Downloading	\	\	取消所有更新和下载， 成功返回 true，HasInited 返回 true 才能调用。
RemoveAllData	\	\	删除所有数据，成功返回 true，HasInited 返回 true 才能调用。如果数据为更新中或更新暂停，执行完毕后数据状态为 DownloadStatus::HasUpdate 如果数据为除未下载外的其他状态，执行完毕后数据状态为 DownloadStatus::UnDownload
Remove OldVersionData	**[in] bool is_delete:** true 移除，false 只检查离线数据是否存在	接口调用方	返回旧版离线数据的大小，不存在为 0。 移除旧版离线数据，同步接口。 注意此接口仅用于删除与当前 SDK 不兼容的老版本数据， 使用此 SDK 下载的数据，请使用其他 Remove 接口删除。
GetDownload DataItem	**[in] int32_t download_id:** 行政区号	客户提供参数	根据数据下载 id 获取 DownloadItem，HasInited 返回 true 才能调用，同步接口。 GetDownLoadDataItem 表示的是 CheckNewVersion 调用后的状态，若是单独调用 GetDownLoadDataItem 仅表达上次 CheckNewVersion 调用的状态，不代表当前的状态。
	[out] DownloadDataItem& item	腾讯提供参数	
GetDownload DataItem	**[in] const std::string &district_id:** 行政区号	客户提供参数	根据区域 id 获取 DownloadItem， 同步接口，HasInited 返回 true 才能调用，同步接口。
	[out] DownloadDataItem& item	腾讯提供参数	
HasDownload	**[in] const std::string &district_id:** 行政区号	客户提供参数	数据是否下载，同步接口，HasInited 返回 true 才能调用。 根据 districtId 获取数据是否已经下载，数据状态为 Downloaded、PartDownloaded（省数据）、HasUpdate、UpdateWait、Updating、UpdatePause 时返回 true。
GetCountry DataVersion	**[out] std::string &** **data_version:** 数据版本	腾讯提供参数	获取当前配置文件的版本，HasInited 返回 true 才能调用， 同步接口。
CheckData	**[in] std::weak_ptr <DataCheckListener> listener:** 数据扫描监听器	客户提供参数	扫描本地数据。扫描数据会更新省市数据的状态和大小， 可通过 GetDataTable 获取新的省市列表。

Cancel CheckData	\	\	取消检查数据。CheckData 耗时可能会比较长 （特别是如果下载的城市比较多） 如果 CheckData 被终止，那么会 OnCanceled。
SetTest Environment	**[in] bool enable:** 设置为 true 后，数据配置文件将从测试服务器拉取	客户提供参数	设置测试环境，同步接口。 设置为 true 后，数据配置文件将从测试服务器拉取。 内部使用，外部调用者无需关心
	[in] std::string url: 配置文件的 url		

【输出信息说明】

监听器	接口	参数	含义
DownloadListener	OnInitFinished	\	初始化完成回调
	OnDownloadProgress	**uint32_t download_id:** 下载城市的行政区号	下载进度回调
		int32_t progress: 下载进度 [0,1000]	
	OnUpdateProgress	**uint32_t download_id:** 下载城市的行政区号	更新进度回调
		int32_t progress: 下载进度 [0,1000]	
	OnDownloadStatusChanged	**uint32_t download_id:** 下载城市的行政区号	下载状态变化回调
		DownloadStatus status: 下载城市的状态	
	OnDownloadFinished	**uint32_t download_id:** 下载城市的行政区号	下载完成回调
	OnErrorMessage	**uint32_t download_id:** 下载城市的行政区号	下载/删除过程中出现的异常状态信息回调
		const mapbase::RetMessage &msg: 错误原因	
UpdateListener	OnNewDataTable	**const std::vector<DownloadDataItem> &new_items：** 新的城市列表 **const mapbase::RetMessage &msg：错误信息**	数据检查更新回调，当 msg.ret_不为 0 时，意味 CheckNewVer 出现错误，请处理错误后再进行其他动作，成功时 new_items 返回城市列表

【示例说明】

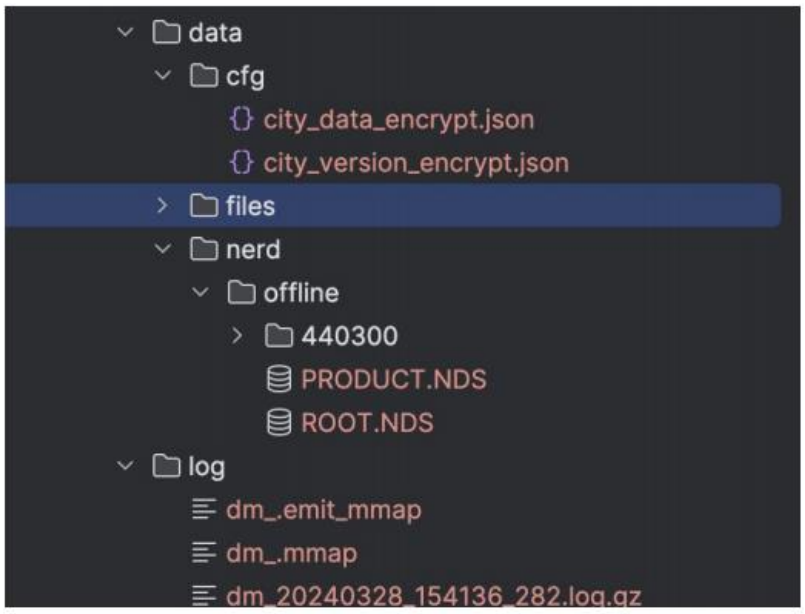
数据下载示例：

```
auto http_impl_ptr = std::make_shared<HttpImpl>();
auto download_listener_ptr = std::make_shared<offline_data::DownloadListener>();
auto update_listener_ptr = std::make_shared<offline_data::DataUpdateListener>();
offline_data::Setting setting;
setting.data_path = "./data"; // 数据路径
setting.log_path = "./log";  // 日志路径
setting.log_level = offline_data::LOG_LEVEL::ERROR; // 日志等级
setting.channel = "10000100102"; // 渠道号
setting.imei = "1234567890a"; // 设备唯一渠道号
setting.log_level = offline_data::LOG_LEVEL::INFO;
setting.log_file_prefix = "tencent_datamanager_"; // 日志前缀
setting.log_file_size = 4; // 单个日志文件大小上限，单位MB
setting.log_total_size = 64; // 日志目录总大小，单位MB
setting.log_save_time = 24 * 30; // 日志默认保存时间，单位小时
setting.use_self_cert_domain = true; // 使用自签名域名时请在注入的http_interface接口实现中配置ca证书

auto offline_service = offline_data::OfflineDataService::Create(setting,http_impl_ptr);
offline_service->SetDownloadListener(download_listener_ptr);
offline_service->Init();
offline_service->CheckNewVer(update_listener_ptr); // 必须调用checkeNewVer成功之后才能下载数据
offline_service->RequestDownload(100000); // 传入行政区号进行下载
offline_service->CheckRegionsData(); // 下载更新全国行政区数据
```

具体详细接口见附录三：离线数据管理引擎接口文档

输出目录示例：



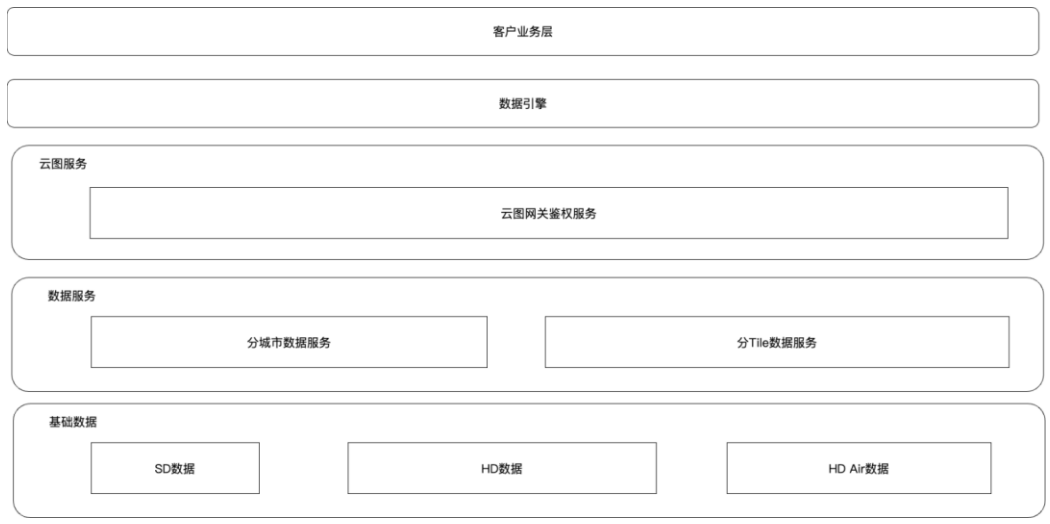
【异常说明】

错误码	含义
const int HttpTimeOutError = 10001;	网络超时
const int HttpUnknownError = 10002;	http 未知错误
const int HttpCancelError = 10003;	http 取消错误
const int AuthError = 30001;	鉴权错误
const int RequestConfigError = 30002;	请求配置文件服务错误
const int ConfigResponseError = 30003;	配置文件服务返回错误
const int ConfigMD5Error = 30004;	下载的配置文件 MD5 校验失败
const int WriteFileError = 40001;	curl 写入文件失败
const int UnzipFileError = 40002;	解压文件失败
const int DataFileMD5Error = 40003;	离线数据文件 MD5 不一致
const int DeleteFileError = 40004;	删除文件发生错误
const int CreateDirError = 40005;	创建文件夹发生错误

const int CopyFileError= 40006;	拷贝文件发生错误
const int MoveFileError= 40007;	移动文件发生错误
const int CancelUpdateOtherCityDataFirst = 6001;	请先取消城市数据更新
const int CancelDownloadOtherCityDataFirst = 6002;	请先取消城市数据下载
const int RemoveOtherCityDataFirst = 6003;	请先删除其他城市数据

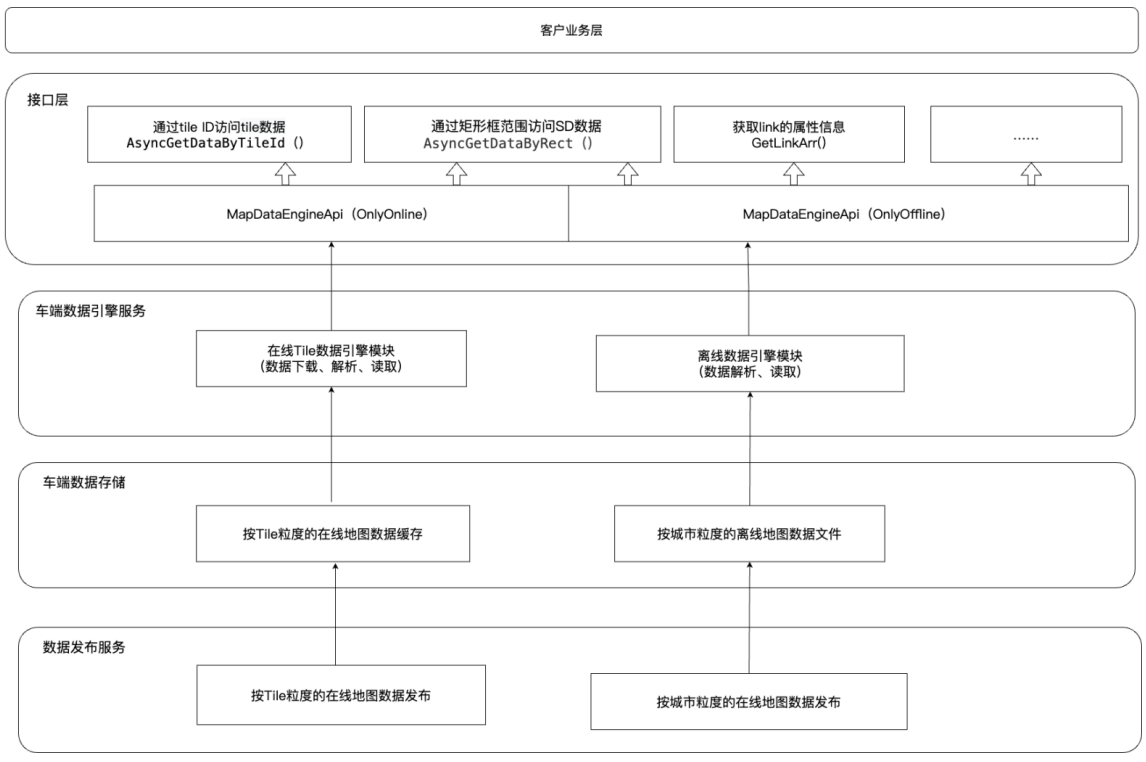
3.3.3 地图数据发布和更新服务

地图数据服务包含了分城市数据服务、分 tile 数据服务，主要是服务于车端 AD 域的自动驾驶地图数据更新的工具；



3.3.3.1 数据更新功能模块架构

腾讯地图数据服务支持按 Tile 级别和城市粒度发布地图数据。车端系统能够按 Tile 进行在线数据缓存，同时也支持按城市粒度的离线数据缓存。这些数据由车端数据引擎服务进行解析与读取，便捷地提供给客户使用。

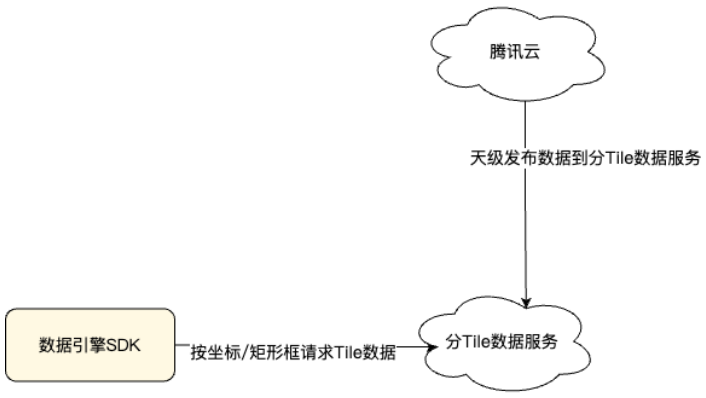


当前数据服务产品如下表所示：

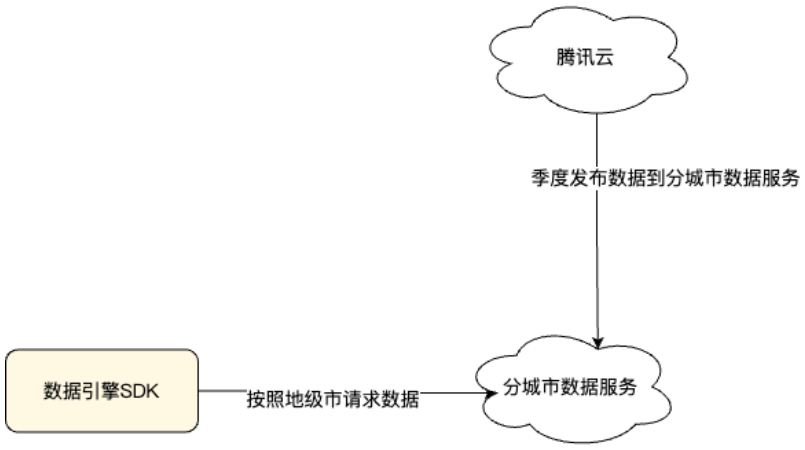
数据服务	更新频率	数据类型
分 Tile 数据服务	天级	SD 数据
	月度	SD 数据、HDAir 数据、HD 数据
分城市数据服务	季度	SD 数据、HDAir 数据、HD 数据

目前数据引擎与数据服务的关系：

（1）按 Tile 粒度的在线地图数据服务：腾讯地图数据天级/月级别发布数据，数据引擎可以按照坐标/矩形请求 Tile 数据；目前支持天级别、月级别的更新频率；



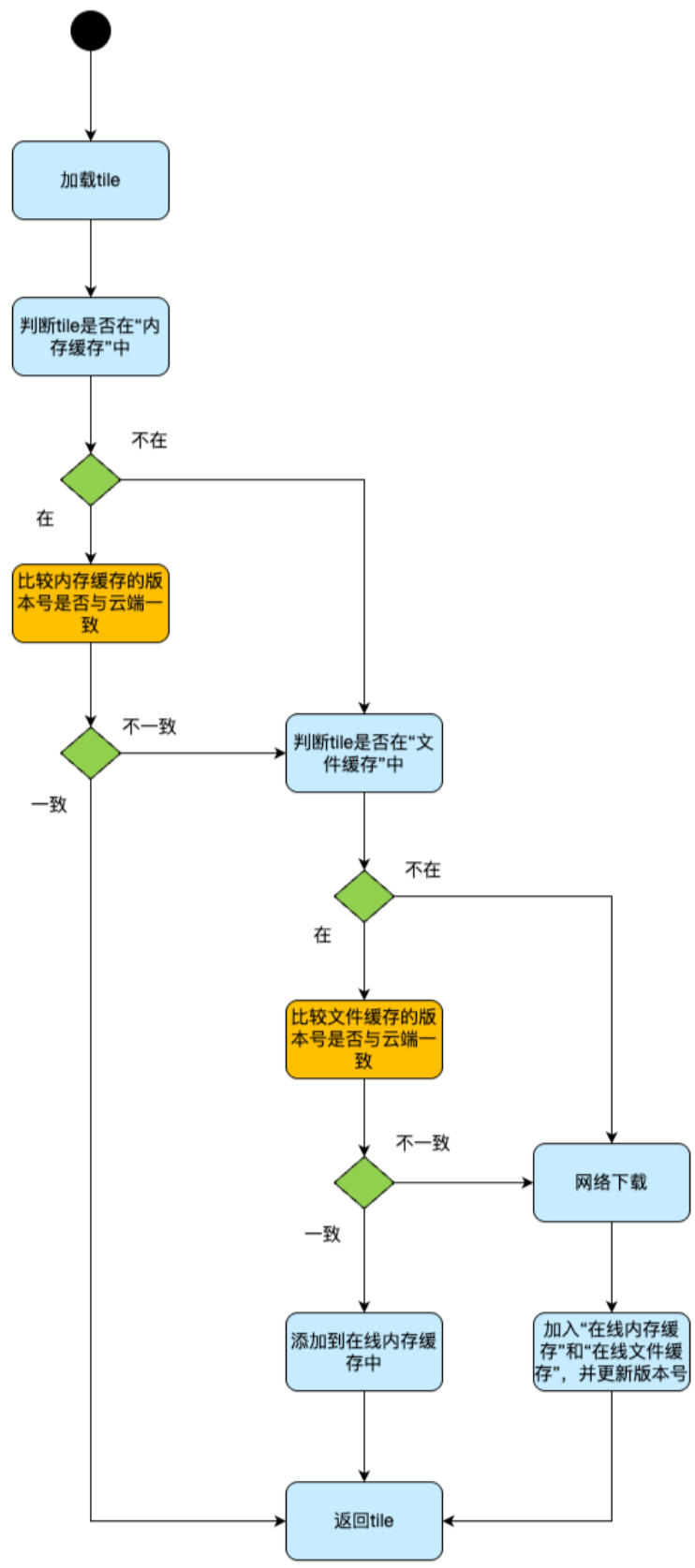
（2）按城市粒度的地图数据服务：腾讯云季度发布数据，数据引擎可以按照地级市进行请求数据；



3.3.3.4 车端数据更新流程

3.3.3.4.1 车端调用在线 tile 更新及访问流程（在线 tile 下载及更新在引擎内部完成）-tile 服务支持天级更新/月底更新：

客户按预先算好的导航路线的轨迹坐标去请求数据，在线访问 tile 数据时，引擎每隔 60s 会更新一次 tile 的云端支持的版本号，并保存，数据版本会更新到与云端一致；



（1）当 tile 在内存缓存中，如果缓存版本号和云端同步的不一致，那么引擎进入第（3）步，如果版本号一致则返回 tile；

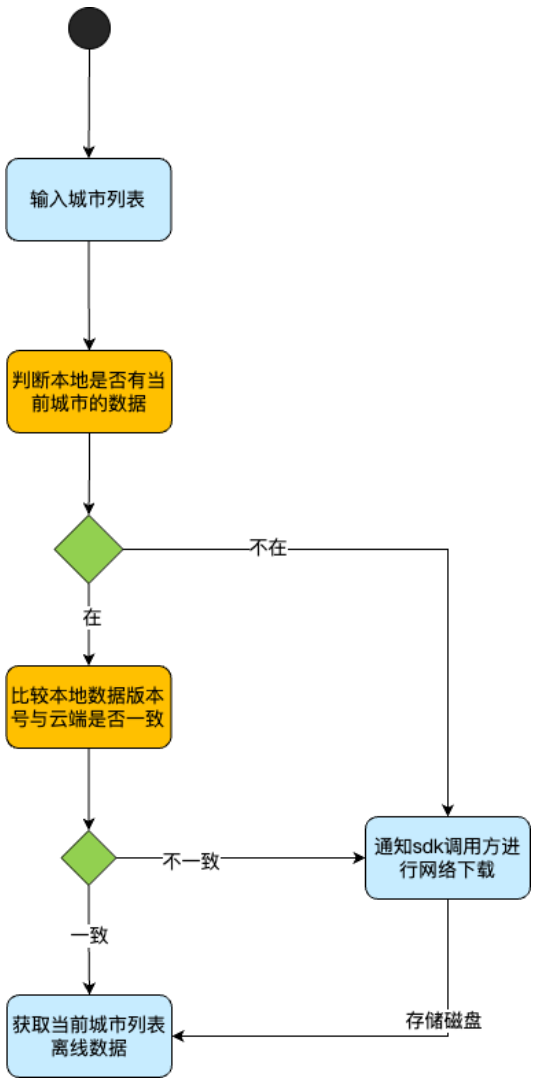
（2）如果目标 tile 不在内存缓存中，进入第（3）步；

（3）判断 tile 是否在文件缓存中，如果在文件缓存中，进一步比较缓存的版本号和云端同步的版本号，相同则返回 tile，不相同则进入第（4）步；如果不在文件缓存中，进入第（4）步；

（4）发起网络请求拉取数据，更新版本号，并将其加入内存缓存和在线文件缓存中。如下图所示。

3.3.3.4.2 车端调用分城市数据服务-季度更新

客户按预先算好的导航路线的轨迹坐标获取城市列表，在使用离线数据管理引擎（Data Manager, DM）时，通过请求分城市数据服务获取数据；



（1）当前城市在车端磁盘中，如果数据版本号和云端同步的一致，那么引擎直接获取当前城市列表的离线数据；

（2）当前城市在车端磁盘中，如果数据版本号和云端同步的不一致，则需要获取当前最新版本的离线数据；

（3）当前城市不在车端磁盘中，则根据提提供的城市信息进行下载，缓存到存储磁盘中，进行获取使用；

3.3.4 使用场景示例-常见使用场景

场景一：座舱导航算好路线之后，将经过异源或者同源匹配过的路线 link id 串和形点传递到自驾域，通过 link 上的形点计算出当前 link 所在的 tile id（参考 3.3.1.3.12 ）；然后调用 AsyncGetDataByTileId（参考 3.3.1.3）读取 tile 数据块，再调用 GetLinkByID（参考 3.3.1.3.7）或者 GetLinkByRawID（参考 3.3.1.3.8） 接口查询对应的 link 信息；

场景二：座舱导航算好路线之后，将经过异源或者同源匹配过的路线 link id 串和形点传递到自驾域，然后调用 AsyncGetDataByRect 接口查询周边范围内的地图数据，再调用 GetLinkByID（参考 3.3.1.3.7）或者 GetLinkByRawID（参考 3.3.1.3.8） 接口查询对应的 link 信息；

3.3.5 地图数据 SDK 常见问题 (2024 年 12 月 23 日新增)V1.3

3.3.5.1 获取不到数据怎么办？

首先检查 tileId 的层级是否正确，SD 数据的 tileId 为 13 级，HD 数据的 tileId 为 15 级，BMD 数据的 tileId 可以是 3、5、7、10、13、15 级。

再检查初始化设置的引擎在/离模式是否正确，在线设置 nerd::api::DataPreference::kOnlineOnly，离线设置 nerd::api::DataPreference::kOfflineOnly。

再检查离线模式的离线文件路径是否正确，LANE.NDS 存放路径 GlobalConfig.data_root_path/lane/LANE.NDS，BGPOI.NDS 存放路径 GlobalConfig.data_root_path/BGPOI.NDS。

3.3.5.2 引擎返回的数据坐标怎么使用?

引擎返回的是 NDS 规范的球面坐标，结构为 Coordinate，可以通过 Coordinate::ToGeo()转换为 gcj02 坐标。

3.3.5.3 从导航 SDK 获取的 SD 路线 ID 如何调用 nerdapi sdk 获取 link 数据？

导航 SDK 获取的 linkID 是 rawLinkId。

需要先取到 rawId 对应的 tile 或者 rect 数据，之后调用 GetLinkByRawID 取对应 link。

获取数据有两种方式：

方式 1. 使用 AsyncGetDataByTileId 接口，沿路根据行点取 tile，比如说从起点开始取 tile，取到 tile 之数据后，遍历路线的 rawLinkId,使用 GetLinkByRawID 接口，这个 tile 内的 raw linkId 对应的 link 都可以找到，出现找不到的就需要根据路径往后取 tile，再重复这个过程；

方式 2. 使用 AsyncGetDataByRect 接口，从起点开始取一个路线矩形框，遍历路线的 rawLinkId，这个框内路线 link 可以使用 GetLinkByRawID 取到，再移动矩形框继续去沿途的 link，需要注意矩形框不要太大。

3.3.5.4 是否有通过坐标获取所在道路 ID 的接口，如果没有，可以通过什么方式间接获取？

没有。可以用 AsyncGetDataByRect 获取矩形范围内的 link 集合，再从 link 集合中选择离指定坐标最近的 link。

3.3.5.5 怎么获取全局规划的 linkid 集合？

从导航 SDK 拿规划路径的 linkid 集合，从 nerdapi 拿不到全局规划的 linkid 集合。

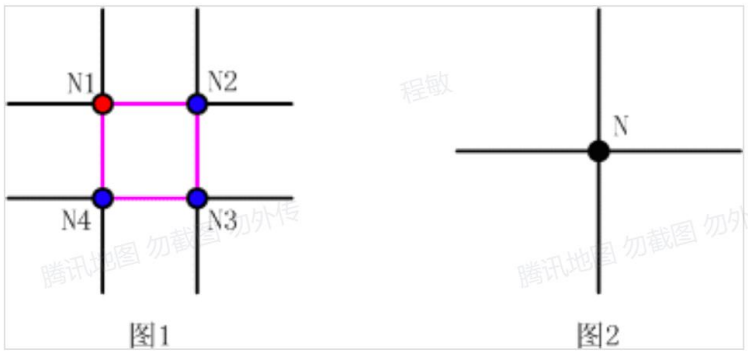
3.3.5.6 ILinkNode 是否对应的就是路口？

LinkNode 是 link 的端点，link 在很多场景会打断，不一定是路口，比如一个端点可能只有一个入边一个出边（没有分歧）。

3.3.5.7 主 Node 和子 Node 能否提供示意图帮助理解，主 Node 和子 Node 之间是否也有一条 link？

图 1 为复杂路口，一般有上下行分离 link 构成，路口有 4 个 Node，其中一个为主 Node，用来标识这个路口。

图 2 为简单路口，一般由非上下行分离 link 构成，路口只有一个交叉点。



3.3.5.8 两个 Node 之间是否有且只有一条 link？

是的。

3.3.5.9 双向道路的是对应一个道路 id 还是两个道路 id？如果是两个道路 id 是否有提供正反向道路的关联关系？

一条双向通行的 link 只有一个 Id

3.3.5.10 道路是否能区分主/辅道路，如果能区分是否有提供主道路与辅道路的关联关系？

可以区分主辅路，有主辅路属性，主辅路无关联关系，需要通过连通和位置关系判断

3.3.5.11 路口内道路是否有转向信息，例如直行，左转，掉头等？

有转向信息，路口处 link 的车信表达

3.3.5.12 一条 link 的车道数量是否是固定的？对于快驶入路口处的车道扩增导致车道数量变化怎么表达？

一条 link 上的车道数是固定的，在高精数据覆盖的区域车道数变化位置会打断 link，普通 link 的车道数不一定准确，可能是在整个 link 占比较多车道数。

3.3.5.13 link->GetGeometry()取的 link 行点方向？

GetGeometry 是从起点到终点方向的行点列表

3.3.5.14 link 打断规则

部分车道数变化位置、道路等级变化，道路类型变化、通行状态变化、道路名称等数据属性存在变化的场景会打断

3.3.5.15 SD 数据是否可以获取路口处车道连通关系？

SD 数据没有车道连通关系，可以获取路口处 link 的车信信息，车信信息中有车道可通行的方向

3.3.5.16 如何获取 link 的车信？

link->GetGuidance(nerd::api::GuidanceType::kLaneMetaInfo)，这样可以取到 link 的车信

3.3.5.17 SD 数据 linkID 和 rawLinkId 有什么区别？

linkID 是 tileId+tile 内索引组成，rawLinkId 为 link 原始 id

3.3.5.18 坡度和曲率是什么级别的？

坡度，曲率是 link 级别的，不是 lane 级别的，不一定每条 link 都有坡度、曲率

3.3.5.19 车道数量变化会切分 link 吗？没有高精地图的地方，车道数量的精度有保障吗？

没有高精的地方车道数变化不会打断 link，赋值 link 段内长度占比较多的车道数

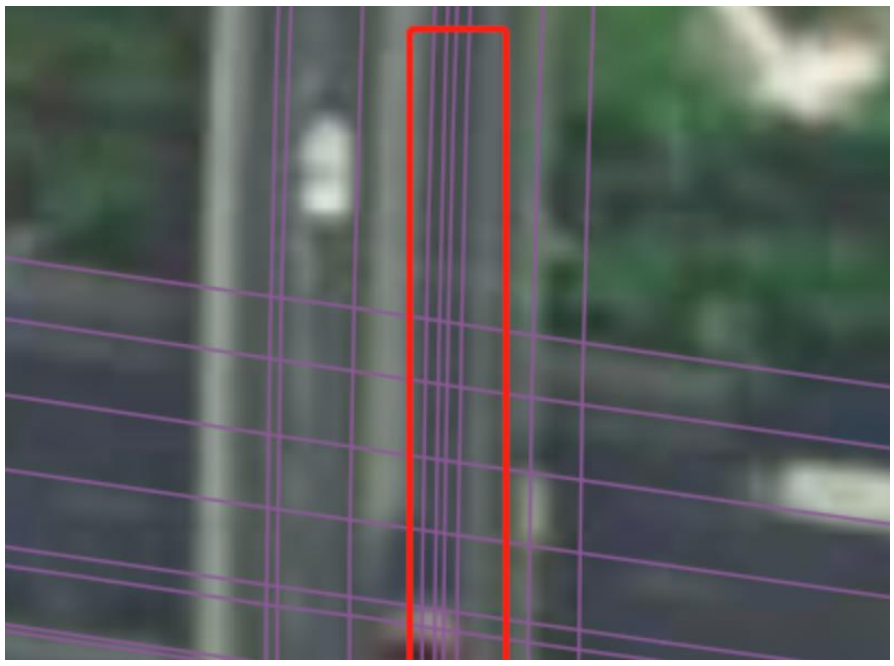
3.3.5.20 如果当前 lane 的 nextLane 跨 tile 了，如何获取？

next 是跨 tile 的时候，需要先获取 NextlaneId， 根据 NextlaneId 里面的 tileID， 再去获取相应的 tile 数据，之后再获取 next， preLane 同理。

3.3.5.21 HDmap 中提取到的 link 数量要比 SDmap 中的 link 数量少很多，但是 HDmap 中的 link 拓扑连接更加丰富。这样理解是对的吗？

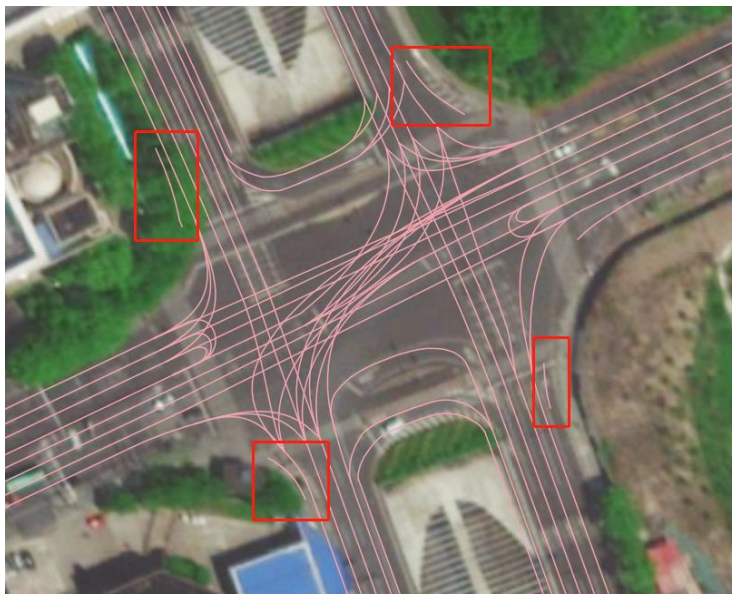
hd 比 sd 少时因为覆盖范围不一样，hd 跟 sd 的连通表达方式不太一样，模型不一样，sd 是用交叉口内部路，高精是单独的转向 link

3.3.5.22 为什么下图中有些地方的 boundary 间隔特别小？



这个四条线分别归属上下行两个方向，每个方向两条线，分别是道路边缘和路肩线

3.3.5.23 为什么下图中 lane 中心线附近有些线是孤立存在的？



Lane 中心线的情况，最外侧的那个是车道中心线是路肩的车道，车道的属性是路肩，本身路肩属性的车道在现实世界中是不连续的，所以存在不连通的情况。

3.3.5.24 SD Map 的 link 里是否有路口转向车道的顺序排列

通过 link->GetGuidance(nerd::api::GuidanceType::kLaneMetaInfo);可以取到车道转向信息

3.3.5.25 GetGuidance()返回的车道通行信息中，有个 arrow，这个是怎么组合在一起的，如果 link 中有多条车道，怎么表示转向信息的？

LaneTrafficInfo 是个数组，表示多个车道。每个车道的箭头是由 LaneArrowType 与运算得到的，比如直行+右转的 arrow = KLaneStraight | KLaneRight， 其车道是从左到右的顺序

3.3.5.26 nerdapi 可以获取两点之间的路径吗

nerd 引擎不可以，需用导航 SDK

3.3.5.27 84 转 02 坐标算法是否固定

84 转 02 坐标的算法及参数固定不变

3.3.5.28 Global Config 中的 channel 是什么意思？应该如何指定

渠道号，使用离线数据不用关注

3.3.5.29 LinkAttribute 中的限速 speed_limit_s2e/speed_limit_e2s，为什么会分为从起点到终点和从终点到起点？一条 link 会有两个行驶方向吗？

link 中存在双向道路，双向道路按划线方向，分别记录 s2e 和 e2s 的限速、车道数属性

3.3.5.30 在一条 link 上面，会不会有多个不同的限速？例如 link 前半段限速 30km/h，后半段限速 50km/h

数据中不会有，实际限速位置前后近距离有断点使用已有断点，距离已有断点过远时会打断 link

3.3.5.31 link 的几何形状返回的坐标点的疏密程度是怎样的？直线和弯道会不会有区别？

转弯位置为确保路线平滑形状点比直线密，但并没有规定必须密度范围是多少

3.3.5.32 LaneInfoMeta 中的车道数 lane_num 和 LinkAttribute 中的车道数 lane_num_sum 是否一样？如果不一样，区别在哪

link 车道数以当前路段内较长的赋值，即如果 link 对应的实际车道数为 3 变 4，3 比 4 长，link 车道数会给 3；车信车道数按路口位置实际车道数赋值，3 比 4 长，4 是路口位置车道数，车信中会给 4.

3.3.5.33 LaneInfoMeta 中的车道数 lane_num 是否和 lane_infos 是绑定的，也就是 lane_infos 的 size 是否等于 lane_num?

lane_num 和 lane_infos 的 size 是一样的

3.3.5.34 boundary 的 width 查出来为什么都是 0？

车道边线目前没有宽度信息，值默认是 0

3.3.5.35 Boundary 的定义是什么？

boundary 是车道的分割标线，在车道数变化，SD 道路断点，路口等位置会以道路面整组打断，沿道路方向存在属性变化时不打断，以区间形式返回，相同区间存在多个属性时，代表为多根线表达的分割线，如双白实线，双黄虚线等

3.3.5.36 Boundary 区间的划分原则是什么？以及为什么一个 boundary 区间的属性要用 vector 来存储，例如 BoundaryMarkingTypeVector

boundary 上的可能有多段不同的边线类型，比如前一段的标线是实线，后一段是虚线，因此 boundary 边界区间用 std::vector<BoundarySetV2>。BoundarySetStyle 里使用 vector 是因为同一段边线有可能有多种类型，比如双实线，boundary_types 就是 [MARKING,MARKING]

3.3.5.37 双实线的 boundary_types 是[MARKING, MARKING]，是否表示腾讯地图是把双实线这种复合线型，拆成两条 line 进行标注的？目前有无方法查询单条 line 的数据？

双实线等多线组合表达含义的线在数据标注时以一条线的形式表达，分别两条真实线的属性，坐标位置在两条线的中间，同区域段只有一个类型、颜色的线是单线

3.3.5.38 如果一个 link 正好在 tile 的边缘，那么通过 link 获取其前后继 link，是不是存在获取不到的情况

是的，可以往后探路径上的 tile，取到后面的 tile 再比较

3.3.5.39 LinkAttribute 中的速度等级 speed_class 代表什么？和限速有什么区别？

speed_class 代表顺向限速的限速区间

3.3.5.40 通过矩形框取数据时，矩形框的坐标是否是火星坐标系经纬？

是的

3.3.5.41 矩形框的常规尺寸需要选择多大？

1～2 公里的范围

3.3.5.42 是否有火星坐标系的 GeoCoordinate 类成员转为 ENU 坐标系的接口?

火星坐标系就是 gjc02 坐标系

3.3.5.43 是否存在火星坐标系的 GeoCoordinate 类成员 向 Coordinate 类成员转换的接口？

导航会下发导航路径的 linkId 列表(rawId)，取沿途的 sd 数据，通过 GetLinkByRawID 取到对应的 sd link，再到 link 中取 GetLaneGroupIds，根

据 LaneGroupId 到 hd 中取高精数据。

gjc02 转 NDS 坐标用 FromGeoCoordinate。

3.3.5.44 Link 通过 GetLinkCurvature 接口获取的曲率和通过 GetLinkPosition 获取的曲率有何不同？

GetLinkPosition 已经废弃

3.3.5.45 Link 通过 GetGeometry 接口获取的坐标点的个数和什么相关？

link 几何是按照道路实际形状进行绘制，点间隔只有最近距离的限制，形状点之间的距离不一致。

3.3.5.46 link 获取的曲率正负如何定义，和车道方向是否相关？

沿着 link 的行驶方向，向左侧弯曲是正，向右侧弯曲是负

3.3.5.47 Link 车道数目是否包含应急车道?

不包括应急车道

3.3.5.48 如何获取导航服务下发路径的 linkid list 查找 HD 数据中的 LaneGroup

导航下发的路径 linkId 列表中 Id 最低位表示方向，0 表示从 link 的起点到终点方向，1 则相反。linkId 右移 1 位后得到 rawId，取沿途的 sd 数据，通过 GetLinkByRawID 取到对应的 sd link，再到 link 中取 GetLaneGroupIds，根据 LaneGroupId 到 hd 中取高精数据

四、 车端 SDK 运行环境

操作系统：Linxu，qnx，Android

车端 SDK 可支持主流的各类平台和操作系统：

芯片架构：Arm，x86

硬件平台：orin 系列平台，高通平台，地平线 J3/J5/J6e 等；

注：需要客户提供对应的编译工具链或者 dock 镜像

附录一 异源匹配 SDK 接口文档

异源匹配 SDK 接入文档

附录二 数据引擎接口文档

nerdapi_SD 文档 01-29(3).pdf

附录三 离线数据管理引擎接口文档

离线数据管理引擎使用文档 3.0_1015.pdf

附件四 异源匹配结果和数据引擎联合对接

异源匹配结果和数据引擎联合对接.pdf

附件五 数据引擎集成说明

数据引擎集成说明：nerd-api 集成使用说明 0816.pdf

附件六 license 的激活流程

lic 激活流程：腾讯智驾云图 license 总体方案.docx